AUTHENTIQ

DIGITAL NOTARY / IP PROTECTION

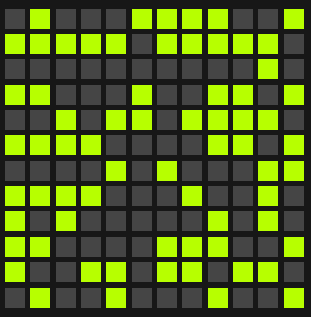
05
PARTS

41
CHAPTERS

20
RANKED FEATURES

THE COMPLETE HANDBOOK

Every feature, every role, every flow, and the entire technology stack behind the product.



OWNERSHIP DNA / SHA-256 DERIVED
479FBE002C4D2DEF0D0A3F12A0AC399B

EDITION 1.0
NOVEMBER 2026

EVIDENCE, NOT ASSUMPTION.

Generated from the running implementation. No invented metrics.

BEFORE YOU BEGIN

Read me first

This is the complete operating manual for AUTHENTIQ: what the website is, what every screen does, what each role can and cannot do, how a real session unfolds from a first visit to a signed-in workspace, and which technology runs behind every button.

THIS DOCUMENT IS	THIS DOCUMENT IS NOT
A description of the code and interface that exist in this workspace today.	A pitch deck, a roadmap, or a promise about features that are not built.
A role-by-role guide to permissions that the backend actually enforces.	A claim that registration creates a legal copyright or a court-admissible judgment.
An A-to-Z map of the frontend, backend, storage, cryptography, ledger, and documentation stack.	A claim that the default ledger is a decentralized public blockchain.

The three sentences to remember

01 AUTHENTIQ turns a file into a cryptographic fingerprint, binds that fingerprint to a secret commitment, and records the result in an append-only history.

02 A zero-knowledge proof shows that the owner knows the opening of the commitment without publishing the file, the secret, or the personal identity behind it.

03 Everything you see in the interface is produced by a real backend operation. Where a capability depends on missing infrastructure, the product says so instead of inventing a result.

THE HONESTY RULE

No fake success states, no hardcoded statistics, no pretend blockchain transactions. The workspace ships empty except for three demo login accounts. Every number on a dashboard is computed from records the user is allowed to see.

Where the facts come from

Every version number, limit, route, endpoint, and permission in this handbook was checked against the running project: the FastAPI application, the React route tree, the SQLAlchemy models, the pytest suite, and the documentation in the repository. The product currently targets a hosted PostgreSQL database (Neon) for relational state, with encrypted files and the ledger on local disk. That split is explained in Part IV. It is written for founders, operators, creators, reviewers, and engineers who need the whole system in one place. The repository at the project root and the live API at /api/docs are the source of truth.

If a sentence in this handbook and a sentence in the code disagree, the code wins. That is the point of documenting a working system rather than an imagined one.

DOCUMENT MAP

What is inside

Five parts, forty-one chapters, one continuous argument: the product, the proof, the pages, the stack, and the boundaries.

PART	CHAPTERS	WHAT IT ANSWERS
I. Orientation	1 - 5	What AUTHENTIQ is, what the website does, what a visitor can do, the complete journey, and who is allowed to do what.
II. Features, ranked	6 - 25	Every feature from the strongest differentiator to the supporting infrastructure, with what it is, why it matters, how it works, and its limits.
III. Page by page	26 - 27	Every public route and every protected workspace page, including what appears on screen and which action belongs there.
IV. Technology A to Z	28 - 36	Architecture, frontend, browser storage, backend, data, cryptography, blockchain, files, PDFs, infrastructure, and configuration.
V. Trust and operations	37 - 41	Security boundaries, the API map, how to run the product, honest limitations, a verification checklist, and a glossary.

Three ways to read it

THE FOUNDER PATH	Chapters 1 to 5, then the ranked feature chapters 6 to 25. Stop at chapter 40 for the honest limits. This is the shortest route to understanding the product and its boundaries.
THE ENGINEER PATH	Chapter 28 for the architecture, then chapters 29 to 36 for the stack, then chapter 37 and 38 for security and the API. Use chapter 41 to reproduce the important claims.
THE REVIEWER PATH	Chapters 5, 8, 33, 34, 37, and 40. This covers roles, the trust score, cryptography, the ledger, the trust boundaries, and the limitations without reading every screen.

How the features are ranked

The ranking in Part II is a product ranking, not a measure of lines of code. Features are ordered by four tests: how much unique trust the feature creates, how hard it would be for a competitor to copy honestly, how deeply other features depend on it, and how often a real user touches it. A feature can be foundational and still rank lower if the user never sees it directly. Security controls, for example, are essential but deliberately quiet, which is why they appear near the end of the list rather than at the top.

A NOTE ON NUMBERS

Exact counts in this handbook refer to the current implementation: 76 backend tests, 30 application tables, 20 ranked features, 29 application routes, five roles, and three demo accounts. If you add code, re-run the checks in chapter 41 before trusting these numbers again.

The table of contents that follows lists every chapter with its page number. Part openers repeat the chapter list so you can navigate a printed copy without flipping back.

CONTENTS

Parts, chapters, and the page where each begins.

SECTION

PART I ORIENTATION 6

01 What AUTHENTIQ is 6

02 What the website does 8

03 Visitors: the public surface 10

04 The complete A-to-Z journey 11

05 Accounts, roles, and permissions 14

PART II FEATURES, RANKED 19

06 Zero-knowledge ownership proofs 20

07 Canonical fingerprint and chunked integrity 22

08 Append-only ledger and Integrity Center 23

09 Evidence packet, trust score, and Ownership DNA 24

10 Registration and PDF certificates 25

11 Silent sharing and public verification 26

12 Encrypted idea vault 27

13 Duplicate and similarity detection 28

14 Dispute resolution workflow 29

15 Two-party ownership transfers 30

16 Team projects and unanimous ownership 30

17 Creative timeline and versions 31

18 AI provenance declarations 32

19 Append-only audit trail 33

20 Notification inbox 34

21 Dashboard and activity heatmap 35

22 Global search 36

23 Settings, sessions, and organization controls 37

24 Security controls and rate limiting 39

25 Operations, documentation, and tests 40

PART III PAGE BY PAGE 41

26 Public pages 41

27 Workspace pages 44

PART IV TECHNOLOGY A TO Z 49

28 Architecture at a glance 49

29 Frontend stack A to Z 52

30 Frontend storage and browser state 54

31 Backend stack A to Z 55

32 Data layer and schema 57

33 Cryptography 59

34 Blockchain and ledger tooling 61

35 Files, PDFs, and QR codes 63

36 Infrastructure, documentation, and configuration 64

PART V TRUST AND OPERATIONS 68

37 Security model and trust boundaries 68

38 API map, status codes, and limits 71

39 Running AUTHENTIQ 77

40 Honest limitations 79

41 Verification checklist and glossary 81

PART I

ORIENTATION

What the product is, who it serves, and how a session unfolds from first visit to signed-in work.

Read this part before the feature catalogue. It defines the vocabulary used everywhere else: fingerprint, commitment, proof, ledger, share link, and the five roles.

IN THIS PART

01 What AUTHENTIQ is
02 What the website does
03 Visitors: the public surface
04 The complete A-to-Z journey
05 Accounts, roles, and permissions

PART I OF V

CHAPTER 01 · ORIENTATION

What AUTHENTIQ is

AUTHENTIQ is a digital notary and intellectual-property protection workspace. It records cryptographic fingerprints of creative work, proves ownership without revealing secrets, and preserves an append-only history of every important action.

41	29	20	24	03
BACKEND TESTS	DATABASE TABLES	RANKED FEATURES	APP ROUTES	ROLES

The problem in plain language

Creative work is easy to copy and hard to date. A screenshot can be edited. A cloud drive timestamp belongs to the provider. An email to yourself is weak evidence and exposes the work. Public blockchain timestamping is expensive, permanent, and often incompatible with privacy: the last thing a writer wants is the hash of an unpublished manuscript or the identity of a client attached to a public ledger forever.

AUTHENTIQ takes a narrower, more useful position. It separates three questions that are usually mixed together: what exact bytes existed, who knew the secret behind the ownership claim, and when the record entered an ordered history. Each question gets its own mechanism, and each mechanism is honest about its limits.

The four questions the system answers

QUESTION	ARTIFACT	WHAT IT ESTABLISHES	WHAT IT DOES NOT ESTABLISH
What exact bytes existed?	SHA-256 fingerprint over the file	The registered content is byte-identical to the file that was hashed.	That the content is original or legally protected.
Who knows the ownership secret?	Poseidon commitment and Schnorr proof	The prover knows the opening of the commitment bound to the registration.	That the prover is the legal author.
When was it recorded?	Ledger record and certificate	The record existed at a point in an ordered, hash-chained history.	Independently witnessed time unless a chain adapter is enabled.
How did ownership change?	Transfers, teams, disputes, audit log	Every accepted change is represented by a new event rather than an overwrite.	A court judgment or a title registry.

What makes it different

- Real cryptography, not a mock. SHA-256, Poseidon-254 over BN254, Pedersen commitments, and non-interactive Schnorr proofs run in Python at request time. The tests tamper with proofs, ledgers, and audit chains to prove the checks are real.
- Privacy by construction. The file never enters a public ledger. Silent-sharing links expose only the fields the owner approved, and the original file is never served by the public verification endpoint.
- Evidence you can carry. A registration produces a PDF certificate, a ZIP evidence packet, a trust score, and an Ownership DNA grid derived from the fingerprint.
- Operational honesty. The default ledger is a labelled local simulation. Polygon Amoy is an optional real testnet adapter. Missing email delivery and MFA are reported as unavailable instead of simulated.

What AUTHENTIQ is not

It is not a law firm, a court, a copyright office, or a production custody service. A registration timestamp is priority evidence, not an automatic legal determination of authorship. The default ledger detects local tampering but is not distributed consensus, and an operator with filesystem access can rewrite it. The default proof proves knowledge of the commitment opening; equality between the presented file and the registered fingerprint is checked outside the proof. These boundaries are repeated in chapter 40 because they matter more than the marketing.

THE PRODUCT IN ONE SENTENCE

Register the exact bytes, commit to a secret, prove you know the opening, and keep the history append-only, while showing only what the owner chose to disclose.

Who it is for

AUDIENCE	WHAT THEY DO WITH IT
Independent creators	Register artwork, writing, music, code, or designs; keep the file private; share a proof link with a client or platform.
Studios and agencies	Create an organization, register client work, coordinate team contributions, and keep an evidence trail for handovers.
Reviewers and platforms	Verify a proof package against a public link without receiving the original file or the owner's identity.
Legal and compliance teams	Export a portable evidence packet and inspect the ledger and audit chain before a dispute.

CHAPTER 02 · ORIENTATION

What the website does

The site has a public marketing surface, a signed-in workspace, and a disclosure-limited verification surface. Each has a different audience and a different security posture.

SURFACE	WHO USES IT	WHAT IT CAN REVEAL	AUTHENTICATION
Landing page at /	Anyone	Product narrative, examples, legal text, and calls to action	None
Account pages	New and returning users	Sign-up, sign-in, password reset, session restoration	None until sign-in succeeds
Workspace	Creators, organization admins, verifiers	Registrations, proofs, teams, transfers, disputes, alerts, audit	JWT access token plus a server-side session
Public verification	Anyone with a link	Only the fields the owner approved, plus a proof result	None, by design

The three evidence layers

The interface never presents a single undifferentiated badge called verified. It separates content evidence, ownership evidence, and history evidence, because they fail in different ways and should be judged separately.

CONTENT EVIDENCE	SHA-256 over the exact file bytes, recomputed by the server after every chunk is verified. The browser also hashes the file locally, but the server hash is authoritative.
OWNERSHIP EVIDENCE	A Poseidon commitment C = Poseidon(fingerprint, secret) and a Pedersen commitment over BN254 G1. The secret is generated on the server, encrypted at rest, and never returned to the browser.
HISTORY EVIDENCE	A hash-chained JSONL ledger for registration and ownership events, plus a separate hash-chained audit log for security-relevant actions.

The lifecycle of one record

01 The browser computes a SHA-256 fingerprint and checks it against existing registrations before uploading anything.

02 The upload is split into chunks. Each chunk is hashed and verified, then the assembled file is re-hashed and compared with the declared overall digest.

03 The server scans the assembled bytes, rejects disallowed extensions, and encrypts the file into an AQVL1 container with chunked AES-256-GCM.

04 The server creates a registration reference, a Poseidon commitment, a Pedersen commitment, and an encrypted commitment secret.

05 File metadata, privacy mode, tags, and the creation timestamp are written to the database.

06 Duplicate and similarity detection runs again against stored records; matches create alerts and notifications for both sides.

07 A ledger event is appended with the record hash, previous hash, and payload digest.

08 The owner can download a certificate, an evidence packet, a trust score, and an Ownership DNA image.

09 Later, the owner can add versions, generate proofs, create share links, transfer ownership, form a team, or open a dispute.

What the server never trusts

- The browser's claimed file hash. The server recomputes it from the assembled chunks.
- The browser's claimed user or role. The token is verified and the user is loaded from the database on every request.
- The browser's claimed ownership. Access helpers check the registration owner, organization membership, or reviewer role before returning data.
- A proof package's internal claims. The verifier checks the signature equation, the nonce, the registration reference, and the fingerprint.

ONE PRODUCT, THREE POSTURES

The marketing page sells the idea, the workspace performs the operations, and the public verification page proves a claim without opening the vault. Keeping those surfaces separate is a security decision, not a design preference.

What the product deliberately does not do

NOT INCLUDED	WHY IT MATTERS	HOW THE PRODUCT RESPONDS
Automatic legal authorship	A hash proves bytes, not personhood or originality.	The UI calls registration priority evidence and says so in certificates.
Public disclosure by default	Most creative work is confidential until release.	Privacy defaults to private; share links are explicit and disposable.
A public blockchain by default	Cost, permanence, and privacy are poor defaults for a prototype.	The local ledger is clearly labelled simulated; Amoy is opt-in.
Email delivery and MFA	Both require external infrastructure and operational policy.	The API reports them as unavailable; development tokens are returned in responses.

CHAPTER 03 · ORIENTATION

Visitors: the public surface

A visitor can understand the product, try a real browser-side hash, create an account, sign in, reset a password, or verify someone else's ownership claim without ever entering the workspace.

What the landing page contains

The landing page at / is a self-contained marketing surface with no data access. It uses the same design system as the workspace: Turret Road display type, Funnel Display body type, an ink and off-white palette, and a single neon-lime accent. The page is long by intention. Each section answers a different objection.

NAVIGATION	Links to the product story, the detection technology, applications, the FAQ, sign-in, and account creation. On mobile it collapses into a menu.
HERO	The core promise, a primary call to action, and a visual language built from fingerprints, proofs, and ledger fragments.
TECHNOLOGY CARDS	Four explanations: SHA-256 fingerprinting, zero-knowledge proofs, the blockchain ledger, and dispute resolution.
INTERACTIVE PREVIEW	A workspace-style demonstration. File hashing runs in the browser with the real Web Crypto API. Proof and verification panels are illustrative and are not connected to the backend; the page does not claim otherwise.
APPLICATIONS	Digital art and graphic design, photography and media, software and source code, research and documentation, and AI-generated content.
FAQ AND LEGAL	Plain-language answers plus terms of use and privacy policy modals. The legal copy states the limitations of priority evidence.
LIQUID CURSOR	A canvas trail that responds to pointer movement. It is disabled for reduced-motion users and touch devices.

What a visitor can do without an account

ACTION	WHERE	RESULT
Read the product story and FAQ	/	No data is read or written.
Hash a file in the browser	/ preview panel	A local SHA-256 appears. Nothing is uploaded.
Create an account	/signup	An individual or organization workspace is created.
Sign in	/login	A session starts and the workspace opens.
Recover a password	/login then /reset-password	A reset token is created. Email delivery is not configured, so the token is returned in the development response.
Verify a shared record	/verify/:verificationId	Disclosure-limited metadata and an optional proof check are shown.

Registration: individual or organization

The sign-up form asks for a display name, email, password, and account type. Choosing organization also asks for an organization name. Every public registration receives the creator role. Choosing organization also creates the organization, makes the new user its owner, and assigns the org_admin role. The password must be at least ten characters. The backend normalizes the email, rejects duplicates, hashes the password with Argon2id, and creates a session.

ACCOUNT TYPE	WHAT IS CREATED	IMMEDIATE CAPABILITIES
Individual	A user with the creator role and no organization.	Register work, prove ownership, create share links, protect ideas, form teams, transfer, dispute, and export evidence.
Organization	A user with the org_admin role, an organization record, and an owner membership.	Everything a creator can do, plus organization registrations and dispute review within the organization.

Sign-in and session restoration

The login page accepts email and password, offers three demo account shortcuts, and can reveal or hide the password. A successful login returns a 45-minute access token and a 14-day refresh token. Both are stored in localStorage, and the user profile is cached beside them for instant rendering. On reload, the app calls /api/auth/me to validate the token; the cached profile is a display shortcut, never an authorization decision.

Repeated failures lock the account temporarily. The login limiter allows eight attempts per IP and email pair in a five-minute window. The API returns 423 when the account is locked and 429 with a Retry-After header when the limiter trips. A forgotten password can be reset with a token; the reset token lasts 30 minutes and all sessions are revoked after a password change.

WHAT A VISITOR CANNOT DO

A visitor cannot browse registrations, read a private fingerprint, download a file, list ledger events, see audit logs, or enumerate accounts. Those routes live behind the protected shell and are enforced by the backend as well as the router.

The public verification experience

A verification link opens /verify/:verificationId without a login. The page first loads the disclosure-approved record, then optionally accepts a proof package. It shows the title only when the privacy mode is public, the fingerprint only when the share was created with show_fingerprint, and the commitment unless the record is private. Expired or revoked links return 410; a link that reached its usage limit returns 409.

CHAPTER 04 · ORIENTATION

The complete A-to-Z journey

This is the practical sequence. Follow it from a blank browser to a registered work, a verified proof, a collaborating team, and an auditable history.

The first ten minutes

01 Open the site at /. Read the hero and the four technology cards. Optionally hash a file in the preview; nothing leaves the browser.

02 Create an account at /signup. Choose individual unless you need a shared organization identity. Use a password of at least ten characters.

03 Sign in at /login. If you want to inspect the product first, use one of the demo accounts; all three workspaces start empty.

04 Land on /dashboard. The counters are zero, the ledger chip shows the active mode, and the activity heatmap is empty. That is expected.

05 Open /register. Select a file, watch the local hash, and let the duplicate check run before the upload starts.

06 Add a title, asset type, description, tags, privacy mode, and creation date. Choose private unless you intend to publish.

07 Complete the chunked upload. Watch each chunk hash, the assembled digest, and the server verification.

08 Submit the registration. Read the result: fingerprint, Poseidon commitment, Pedersen commitment, ledger transaction, alerts created.

09 Download the certificate. Export the evidence packet. Open the trust score and see which factors are still missing.

10 Open /anonymous-verification, generate a challenge, hash the original file locally, generate a proof, and verify it.

Registering a work, step by step

STEP	WHAT HAPPENS	WHAT CAN GO WRONG
1. Choose a file	The extension and size are checked against the allow-list and the 200 MB cap.	415 for an unsupported extension, 413 for an oversized file.
2. Local hash	The browser computes SHA-256 with Web Crypto.	Nothing is uploaded yet, so a failure is local and harmless.
3. Duplicate check	The fingerprint and, for images, a dHash-64 are compared with existing records.	A match is reported with references and timestamps; registration is still allowed.
4. Initiate upload	The server creates an upload with a chunk plan.	An expired or foreign upload cannot be completed.
5. Send chunks	Each chunk is hashed, size-checked, recorded, and written temporarily.	422 for a chunk hash mismatch; the chunk is not recorded.
6. Complete	Chunks are re-hashed, assembled, scanned, and encrypted into AQVL1.	Missing or corrupt chunks reject the upload; a scanner rejection writes an audit event.
7. Register	Metadata, commitments, privacy, and timestamps are stored; the ledger event is appended.	A database or ledger failure surfaces as an error; no partial success is claimed.
8. Detect	Server-side duplicate and similarity checks run again; alerts and notifications are written.	An exact duplicate creates mutual alerts but does not block the registration.
9. Certify	A PDF certificate can be generated on demand with a QR link.	Generation is synchronous and requires the record to be readable by the user.
10. Package	A ZIP evidence packet can be exported with manifest, ledger receipt, digest, certificate, and instructions.	The packet contains no private key and no original file.

Proving ownership and sharing it

Open /anonymous-verification, choose a registration, and request a challenge. The API returns a nonce that expires in ten minutes. Select the original file; the browser hashes it locally. If the hash matches the registered fingerprint, the generate button becomes available. The server decrypts the commitment secret, builds a Schnorr proof bound to the registration, fingerprint, commitment, and nonce, and returns the proof and public signals. Verification checks the equation and the binding, then records the event.

To share without a proof, create a silent-sharing link from the verification page. Choose an expiry between one and 720 hours and a maximum use count between one and 100. The API returns a link such as /verify/abc123 and a proof package. The recipient does not need an account. Single-use semantics require max_uses set to 1; the default is 1.

Working with a team

01 A creator opens /teams and creates a project with a name and an optional description.

02 The manager invites collaborators by email; the invitation is visible to the invited account.

03 Members join and submit contributions. A manager approves a contribution before it counts.

04 The manager proposes ownership percentages. The backend requires an exact total of 100%.

05 Every active member must approve. Once unanimous, the allocation is approved and recorded in the ledger.

06 Any member can download the team certificate. The project retains its contribution and approval history.

Transferring ownership

Only the current owner can start a transfer at /transfers. The recipient must already have an account. The request moves from pending_recipient to recipient_accepted or rejected. If accepted, the owner confirms and the registration owner changes, with a ledger event and an append-only transfer history. Either party can cancel while the request is pending or accepted. Accepted history is never deleted.

Raising and resolving a dispute

Any signed-in user can open a dispute against a registration at /disputes. The current owner becomes the respondent. The system scans for registrations with the same SHA-256 and reports the earliest as priority evidence, explicitly without making a legal ruling. Claimants, respondents, verifiers, and organization admins can attach encrypted evidence. Only a verifier or an organization admin in the registration's organization can resolve or reject, and a resolution summary is mandatory. The evidence report aggregates the records, proofs, versions, transfers, evidence, ledger audit, and audit-chain state.

A RULE THAT PREVENTS CONFUSION

A duplicate or similarity finding does not delete, hide, or automatically invalidate a registration. It creates an alert and a notification so a human can investigate. The earliest registration keeps priority because the ledger is ordered by time.

I want to...

GOAL	OPEN
See my overall status	/dashboard
Register a new file	/register
Browse everything I have registered	/timeline
Inspect one record, certificate, or evidence packet	/registrations/:id
Protect an idea that has no file yet	/ideas
Declare AI involvement or run provenance analysis	/provenance
Collaborate and allocate ownership	/teams
Hand a registration to someone else	/transfers
Investigate copying or challenge a claim	/disputes and /alerts
Prove ownership to a client	/anonymous-verification
Check the ledger, audit chain, and workspace integrity	/integrity and /audit-logs
Read notifications or change my profile	/notifications and /settings

CHAPTER 05 · ORIENTATION

Accounts, roles, and permissions

There are five backend roles and two account types. The role is carried in the token but re-checked against the database and the resource on every request. Route visibility is a convenience, never the security boundary.

The five roles

ROLE	HOW IT IS ASSIGNED	SCOPE	TYPICAL USER
creator	Default for every public registration.	Own registrations, ideas, uploads, shares, transfers, and disputes as a party.	Independent artist, writer, developer, researcher.
org_member	Assigned when an organization invitation is redeemed or a member is provisioned.	Own resources inside the organization plus the shared organization workspace.	Collaborator, staff member, agency contributor.
org_admin	Assigned when an organization account is created, or when an existing admin promotes a member. The creator is also the organization owner.	Own resources plus all registrations in the organization, organization settings, member provisioning, and dispute review inside the organization.	Studio lead, agency owner, research lab manager.
verifier	Seeded demo account; not created by public registration.	Own resources plus all disputes for review and resolution, and the all-ledger scope. Cannot list every registration.	Platform trust and safety reviewer, neutral evidence reviewer.
site_admin	Assigned manually to an operator account. Never created by public registration.	Platform-wide read of users, organizations, registrations, teams, disputes, and the audit trail, plus targeted writes. Still bounded by the API.	Platform operator, incident responder.

Capability matrix

CAPABILITY	CREATOR	ORG ADMIN	VERIFIER	SITE ADMIN
Register, upload, fingerprint	Yes, own	Yes, own and org	Yes, own	Yes, own
Read registration detail and file	Yes, own	Yes, own and org	Yes, own only	Yes, platform read
Generate proof and share link	Yes, own	Yes, own and org	Yes, own	Yes, own
Create and reveal ideas	Yes, own	Yes, own	Yes, own	Yes, own
Create team projects and allocate ownership	Yes, as owner or member	Yes, as owner or member	Yes, as owner or member	Yes, read
Propose, accept, cancel, confirm transfers	Yes, as party	Yes, as party	Yes, as party	Read
Open a dispute	Yes	Yes	Yes	Read
Attach dispute evidence	Yes, as party	Yes, as party or org reviewer	Yes, as party or reviewer	Read
Resolve or reject a dispute	No	Yes, inside the organization	Yes, any dispute	Yes, platform
List all ledger transactions	No, own scope	Yes	Yes	Yes
List every audit event	No, own scope	Yes	Yes	Yes
Rename the organization	No	Yes	No	Yes
Read organization members	Yes, own organization	Yes	No	Yes
Add a member with a generated password	No	Yes, inside the organization	No	Yes
Rotate a member password and revoke sessions	No	Yes, except the owner's own password	No	Yes
Suspend or reactivate an account	No	No	No	Yes
See all registrations in the workspace	No	No, only the organization	No	Yes, filtered

Permissions are enforced in the API. Hiding a navigation item is not an authorization control.

THE ONE-LINE AUTHORIZATION RULE

The backend asks three questions before returning protected data: is the token valid, does the user own the record or belong to its organization, and does the user hold a reviewer role that covers this record? All three are answered from the database, not from the request body.

How an organization adds a person

An organization admin or owner opens Settings, chooses the Organization tab, and fills in a display name, an email address, and a role of Member or Administrator. The backend validates the address, rejects duplicates, creates a real account and an active membership in one transaction, and generates a sixteen-character password with lowercase, uppercase, digits, and a symbol. Only the Argon2id hash is stored. The plaintext password appears once in a one-time sign-in card, and the Copy action places the email and password on the clipboard together.

The new member signs in at the normal /login screen and lands inside the same organization. A Member maps to the creator role and cannot provision other people. An Administrator maps to org_admin and can provision in turn. The owner appears with an Owner badge and cannot be reset by another administrator. The New password action on any other member rotates the credential, clears failed-login counters, and revokes every active session for that member.

Nothing is emailed in this release. The generated password is shown once and is never recoverable; a lost password is replaced by rotation. This is a deliberate trade: a platform that can display a password again is a platform that can leak it again. Every creation and rotation is written to the audit chain and produces an in-app notification for the affected person.

Account types are not roles

An account type describes what kind of identity was created: individual or organization. A role describes what the identity may do. An organization account starts with the org_admin role, while an individual starts as a creator. A verifier can be an individual account. Do not confuse the two columns in the settings page: the interface shows both.

How a session works

- Access token: a JWT that lasts 45 minutes and carries user id, session id, role, and organization id. It is sent as a bearer token.
- Refresh token: 14 days, stored hashed in the session row, and rotated on every refresh. Reuse of an old token fails.
- Session record: created at login and revocable individually or all at once. A password change revokes every session.
- Restoration: the browser calls /api/auth/me during startup. A failed refresh clears local credentials and returns the user to /login with the attempted path saved.
- Logout: revokes the current refresh token and clears local storage. Logout-all uses the same endpoint with a flag.

What a public registrant can do immediately

- Register an unlimited number of creative works, subject to storage and rate limits.
- Keep every upload private, or expose a fingerprint or public metadata through an explicit share.
- Generate real ownership proofs and one-time verification links.
- Protect ideas with encrypted descriptions and Poseidon commitments before any file exists.
- Create a team project, invite collaborators, and propose a unanimous ownership split.
- Add versions, compare fingerprints, and issue version certificates.
- Declare AI involvement and, when provider credentials exist, run provenance analysis.
- Open a dispute, attach encrypted evidence, and export an evidence report.
- Transfer a registration to another account with a two-party acceptance flow.
- Inspect the ledger and audit chain for accessible records and export an evidence packet.
- Change profile details, review active sessions, and revoke a session.

PART II

FEATURES, RANKED

Twenty capabilities, from the defining proof system to the operational floor beneath it. Each chapter follows the same pattern: what it is, why it matters, how it works, what you see, and where it stops.

The ranking is by product leverage, not code size. The proof system is first because it is the hardest claim to make honestly; the operational layer is last because a user should never have to think about it.

IN THIS PART

06 Zero-knowledge ownership proofs
07 Canonical fingerprint and chunked integrity
08 Append-only ledger and Integrity Center
09 Evidence packet, trust score, and Ownership DNA
10 Registration and PDF certificates
11 Silent sharing and public verification
12 Encrypted idea vault
13 Duplicate and similarity detection
14 Dispute resolution workflow
15 Two-party ownership transfers
16 Team projects and unanimous ownership
17 Creative timeline and versions
18 AI provenance declarations
19 Append-only audit trail
20 Notification inbox
21 Dashboard and activity heatmap
22 Global search
23 Settings, sessions, and organization controls
24 Security controls and rate limiting
25 Operations, documentation, and tests

PART II OF V

The ranked list at a glance

RANK	FEATURE	WHY IT SITS HERE
01	Zero-knowledge ownership proofs	The defining trust claim and the hardest capability to copy credibly.
02	Canonical fingerprint and chunked integrity	Every other feature depends on knowing the exact bytes.
03	Append-only ledger and Integrity Center	Turns records into an ordered, checkable history.
04	Evidence packet, trust score, and Ownership DNA	Makes the evidence portable and explainable.
05	Registration and PDF certificates	The core user artifact and the first thing a client sees.
06	Silent sharing and public verification	The commercial use case for the cryptography.
07	Encrypted idea vault	Protects value before a file exists.
08	Duplicate and similarity detection	Turns silent copying into a reviewable event.
09	Dispute resolution workflow	Gives conflicts a structured, authorized process.
10	Two-party ownership transfers	Keeps ownership history correct after a handover.
11	Team projects and unanimous ownership	Models collaboration with real consent.
12	Creative timeline and versions	Preserves the evolution of a work.
13	AI provenance declarations	Adds a dated disclosure layer for AI involvement.
14	Append-only audit trail	Makes account and operational actions reviewable.
15	Notification inbox	Closes the loop between events and people.
16	Dashboard and activity heatmap	Answers what I have and what needs attention.
17	Global search	Makes a growing workspace navigable.
18	Settings, sessions, and organization controls	Identity, recovery, and organization boundaries.
19	Security controls and rate limiting	The foundation that makes the rest trustworthy.
20	Operations, documentation, and tests	The ability to run, inspect, and reproduce the product.

A NOTE ON RANK AND IMPORTANCE

Rank nineteen is not an unimportant feature. Security controls are the floor. They rank low because the user should never have to notice them, not because they are optional.

CHAPTER 06 · CRYPTOGRAPHIC PROOF

Zero-knowledge ownership proofs

This is the defining capability. AUTHENTIQ can demonstrate ownership of a registered work to someone who never receives the work, the commitment secret, or the owner's personal details.

01
OF 20

FEATURE

Zero-knowledge ownership proofs

Prove you know the opening of the commitment without publishing the file, the secret, or your identity.

WHAT IT IS	A non-interactive Schnorr proof over BN254 G1. It proves knowledge of the opening of the Pedersen commitment stored with the registration. The commitment binds a Poseidon digest of the file fingerprint and a random secret.
WHY IT MATTERS	A certificate alone is a document. A proof is a cryptographic answer to a challenge: it is fresh, bound to one registration, and useless to replay. It lets a creator convince a client, platform, or reviewer without handing over the source file.
HOW IT WORKS	• The owner requests a challenge; the API returns a nonce that expires after ten minutes. • The browser hashes the original file locally and sends the digest. • The server compares the digest with the registered SHA-256 and refuses to prove a mismatch. • The server decrypts the commitment secret, builds the proof, and binds it to the registration, fingerprint, commitment, and nonce through Fiat-Shamir. • A verifier checks the Schnorr equation, the binding, the nonce, and the usage rules; the result is stored and audited.
WHAT YOU SEE	The verification page shows the challenge nonce, local SHA-256, commitment, smart signals, a pass or fail result, and recent proofs. The certificate and evidence packet carry the same identifiers.
LIMITS	The default proof establishes knowledge of the commitment opening. Equality between the presented file and the registered fingerprint is checked outside the proof, not constrained inside a circuit. The optional Circom circuit is not wired into runtime.

The proof system is deliberately small enough to audit. It uses pure-Python BN254 arithmetic and a circomlib-compatible Poseidon-254 implementation, with a scheme identifier of AUTHENTIQ-SCHNORR-BN254-G1-V1. There is no trusted setup, no proving key, and no external proving service.

A proof is not a transferable badge. It is generated for one registration and one nonce. Reusing a nonce, changing the fingerprint, or presenting another registration's commitment fails verification. Proof generation is rate-limited to 40 attempts per user per hour, and every attempt is written to the audit trail.

Where to use it

- Send a client a proof link instead of the working file.
- Answer a platform's ownership challenge without exposing an unpublished manuscript.
- Attach a verified proof to a dispute evidence report.
- Raise the trust score by accumulating verified proofs.

READ THE CLAIM PRECISELY

The proof does not say 'this person is the legal author'. It says 'this prover knows the secret that opens the commitment bound to this registration'. That is a narrower, testable, and genuinely useful statement.

CHAPTER 07 · CONTENT EVIDENCE

Canonical fingerprint and chunked integrity

Before ownership means anything, the system has to know exactly which bytes it is talking about. The fingerprint pipeline is the foundation every other feature stands on.

02
OF 20

FEATURE

Canonical fingerprint and chunked integrity

One changed byte produces a different fingerprint. Every chunk is checked before the file is accepted.

WHAT IT IS	A SHA-256 digest over the exact file bytes, computed in the browser for early feedback, recomputed by the server from verified chunks, and stored with the registration, versions, certificates, and ledger payload.
WHY IT MATTERS	A title, a screenshot, or a file name proves nothing. A hash is compact, deterministic, and sensitive to every byte. Chunk verification catches corruption and tampering in transit before an encrypted copy is written.
HOW IT WORKS	• The browser computes SHA-256 with Web Crypto and offers a pre-upload duplicate check. • The upload is planned in chunks of up to 8 MB, with a 200 MB file cap and a 10,000 chunk ceiling. • Each chunk is hashed by the browser and by the server; a mismatch returns 422 and the chunk is not recorded. • On completion the server re-hashes the assembled bytes, compares the overall digest, scans the file, and encrypts it. • The canonical digest is written to the registration, the certificate, the evidence packet, and the ledger payload digest.
WHAT YOU SEE	The registration screen shows the local hash, upload progress, per-chunk verification, the assembled SHA-256, the Poseidon commitment, and the Pedersen commitment.
LIMITS	A hash proves byte identity, not originality, authorship, or legal status. The browser hash is a convenience; the server hash is authoritative. Temporary chunk files are plaintext until completion deletes them.

The upload model has an explicit state machine: initiated, uploading, completed, failed. A registration can only use an upload owned by the same user with status completed. A completed upload cannot be silently reused for a different file because the stored digest is checked again at registration time.

The extension allow-list covers images, documents, spreadsheets, presentations, archives, audio, video, source code, design files, SQL, HTML, CSS, and circuit sources. Anything else is rejected with 415 before storage. The limit is enforced on the server, not only in the picker.

Where to use it

- Prove that a delivered file is byte-identical to the registered master.
- Detect corruption after an upload or a download.
- Compare two versions without reading either file.
- Give a client a compact digest they can verify offline.

CONTROL	VALUE	WHERE IT APPLIES
Maximum file size	200 MB	Upload initiation and completion

CONTROL	VALUE	WHERE IT APPLIES
Maximum chunk size	8 MB	Each chunk request
Maximum chunks	10,000	Chunk plan creation
Hash algorithm	SHA-256	Browser, chunk, assembly, storage, ledger
Encryption at rest	AES-256-GCM, AQVL1	Completed uploads and dispute evidence

CHAPTER 08 · HISTORY EVIDENCE

Append-only ledger and Integrity Center

The ledger is how AUTHENTIQ turns a collection of rows into an ordered history. The Integrity Center is how a human checks that the history has not quietly changed.

03
OF 20

FEATURE

Append-only ledger and Integrity Center

Every important event gets a hash, a link to the previous record, and a page that rechecks the whole chain.

WHAT IT IS

A JSONL ledger with block numbers, previous hashes, canonical record hashes, and payload digests, plus an integrity page that recomputes the chain, the audit chain, storage presence, ledger linkage, duplicate fingerprints, and a workspace digest.

WHY IT MATTERS

A database row can be edited without leaving a trace. A hash chain makes edits visible: changing one record breaks every later link. The Integrity Center turns that property into a readable report instead of a hidden internal check.

HOW IT WORKS

- Registration, proof, transfer, team, and dispute actions append a ledger event.
- Each record includes a block number, previous record hash, payload digest, canonical record hash, actor, entity, and timestamp.
- The file is flushed and fsynced after each append to reduce loss on crash.
- Verification recomputes continuity, previous links, record hashes, and payload digests and reports the first mismatch.
- The Integrity Center also checks that every registration has a ledger record, that encrypted files are present, and that duplicate fingerprints are known.

WHAT YOU SEE

The Integrity Center shows ledger chain status, audit chain status, workspace digest, open issues, last hashes, storage counts, duplicate groups, and a record-level ledger map. The Ledger and audit page exposes raw transactions and audit entries.

LIMITS

This is not distributed consensus. It is tamper-evident and operator-rewritable: someone with filesystem access can rebuild the whole file. Records become independent evidence only when the optional Polygon adapter writes them to a public testnet.

The ledger adapter boundary matters. Application services call append, verify, list, lookup, and status without knowing which implementation is active. The simulation and Polygon adapters share event semantics but have very different trust properties, and the UI labels which one is in use.

The Integrity Center is not a marketing dashboard. It is a diagnostic: if a ledger record is missing, a file has disappeared, or a duplicate fingerprint exists, the page says so. A workspace digest gives you a single value to compare between two moments in time.

Where to use it

- Check the workspace before exporting an evidence packet.
- Show a reviewer that the chain is intact.
- Detect a missing encrypted file before it matters.
- Compare a workspace snapshot after a migration.

THE CORRECT SENTENCE

AUTHENTIQ maintains an append-only local ledger with hash chaining. It is tamper-evident, not tamper-proof, and it is not a blockchain unless the Polygon adapter is configured.

CHAPTER 09 · PORTABLE PROOF

Evidence packet, trust score, and Ownership DNA

Evidence is only useful if it can leave the product. This feature turns a registration into something a client, lawyer, or platform can hold, inspect, and verify offline.

04
OF 20

FEATURE

Evidence packet, trust score, and Ownership DNA

One click produces a self-describing ZIP, an explainable score, and a visual identity derived from the fingerprint.

WHAT IT IS

A ZIP packet containing a canonical manifest, the PDF certificate, a ledger receipt, the file digest, and verification instructions; a 0-100 trust score with visible factors; and an SVG Ownership DNA grid generated deterministically from the SHA-256 fingerprint.

WHY IT MATTERS

A web page is not portable evidence. A manifest with a self-hash can be checked without the server. An explainable score avoids the worst pattern in trust products: a single number with no reasons behind it.

HOW IT WORKS

- The manifest is built from the registration, file metadata, ledger record, proofs, and trust report, then hashed and stored inside itself as manifest_sha256.
- The certificate is generated on demand with the registration identifiers and a QR verification link.
- The ledger receipt carries the block number, record hash, previous hash, and payload digest.
- The file digest is written as a standard sha256sum-compatible line.
- VERIFY.txt explains each step, including how to recompute the manifest hash.

WHAT YOU SEE

The registration detail page offers Certificate, Evidence packet, and Original file. The trust card lists each factor and its points. The DNA panel renders a grid and allows an SVG download.

LIMITS

The packet proves internal consistency and gives you the material to verify it. It does not include the original file or the commitment secret. A trust score is an evidence heuristic, not a legal conclusion or a guarantee of authenticity.

The trust score starts from six positive factors: fingerprint 25, ledger confirmation 25, verified proofs 20, stored file integrity 15, and a perceptual signature 5. Duplicate indicators subtract 5 each, capped at 15. Open disputes subtract 10 each, capped at 20. The total is clamped to 0-100 and graded verified at 90, strong at 70, developing at 45, and unverified below that.

The Ownership DNA grid is not random art. It is derived from the fingerprint bits, so the same file always produces the same pattern. It gives non-technical users something memorable to compare while the hash remains the actual identifier.

Where to use it

- Send a client a packet they can verify without an account.
- Attach a manifest and receipt to a legal or compliance file.
- Explain why a record is not yet verified.
- Compare two printed DNA grids at a glance.

CHAPTER 10 · THE CORE RECORD

Registration and PDF certificates

Registration is the moment a file becomes a record. Everything else in the product either strengthens that record, shares it, moves it, or challenges it.

05
OF 20

FEATURE

Registration and PDF certificates

A durable record of what was registered, when, by whom, and under which privacy mode.

WHAT IT IS	A registration row tied to a completed upload, with a reference, title, description, asset type, tags, privacy mode, creation timestamp, fingerprint, Poseidon commitment, Pedersen commitment, ledger transaction, and status.
WHY IT MATTERS	People need a concrete artifact they can point to. The certificate turns an API response into a document with an identifier, a QR link, and an explicit statement of what the record means.
HOW IT WORKS	• The owner chooses a completed upload and supplies metadata and a privacy mode. • The server derives commitments from the fingerprint and a fresh secret, then encrypts the secret. • Registration, metadata, and the ledger event are written in one flow, with audit entries for the action. • Duplicate and similarity checks run against stored records; matches create alerts and notifications. • Certificates are generated on demand as PDFs with a QR code and a simulation or chain-mode label.
WHAT YOU SEE	The timeline lists records with fingerprints and status. The detail page shows every identifier, the file name and size, the privacy mode, the creation date, alerts created, and download actions.
LIMITS	Registration does not create copyright, prove originality, or guarantee that no one registered the same bytes earlier. The earliest ledger record wins priority, which is why the timestamp and chain matter.

Privacy modes are explicit. Private records are visible to the owner, organization members where applicable, and explicit share links. public_fingerprint may expose the fingerprint and commitment. Public may expose the title and public metadata. The public endpoint never serves the original file or the private witness.

Certificates are not cached on disk. Each download regenerates the PDF from current data, so a revoked or changed record cannot leave a stale certificate being served from a cache.

Where to use it

- Create a dated record before sending a pitch or a client deliverable.
- Give a collaborator a certificate for their portfolio.
- Attach a certificate to a contract or an insurance claim.

CHAPTER 11 · DISCLOSURE CONTROL

Silent sharing and public verification

Most ownership products either keep everything private or publish everything. AUTHENTIQ lets the owner choose exactly how much a verifier learns.

06
OF 20

FEATURE

Silent sharing and public verification

Share the claim, not the file. Every link has an expiry, a use limit, and a disclosure policy.

WHAT IT IS

A verification request with a public identifier, an expiry between 1 and 720 hours, a maximum use count between 1 and 100, disclosure controls, and an optional proof package. The public page needs no account.

WHY IT MATTERS

A client or platform often needs to confirm that a record exists and belongs to someone, without receiving the source file or the owner's email. Silent sharing is the commercial use case that makes the cryptography practical.

HOW IT WORKS

- The owner selects a registration and creates a share with an expiry and use limit.
- The API returns a /verify/:id URL and a proof package.
- The public page loads disclosure-approved metadata only.
- The verifier can submit the package; the API checks the registration reference, fingerprint, nonce, signature, expiry, revocation, and usage count.
- Each successful use increments the counter; 410 means expired or revoked, 409 means the usage limit was reached.

WHAT YOU SEE

The verification page shows the title only for public records, the fingerprint only when enabled, the commitment for non-private records, the ledger transaction, and a clear privacy note.

LIMITS

A link is a bearer token: anyone who has it can use it within its limits. Single-use semantics require max_uses set to 1. Links are not searchable and can be revoked, but they cannot un-disclose information already viewed.

The public endpoint is rate-limited to 30 requests per IP per ten minutes, and share creation to 30 per user per hour. Those limits protect both the verifier experience and the owner's record from enumeration.

The proof package format is AUTHENTIQ-VERIFICATION-PACKAGE-V1. It is JSON, can be pasted into the public page, and contains no secret. The public page reports valid or invalid with a reason, and the result is written to the verification record.

CONTROL	RANGE	DEFAULT
Expiry	1 to 720 hours	72 hours
Maximum uses	1 to 100	1
Fingerprint disclosure	on or off	off

CONTROL	RANGE	DEFAULT
Proof required	optional	optional
Account required to verify	no	no

CHAPTER 12 · PRE-FILE PROTECTION

Encrypted idea vault

Not every valuable creation starts as an upload. The idea vault protects the concept stage with an encrypted description and a Poseidon commitment.

07
OF 20

FEATURE

Encrypted idea vault

Timestamp an idea before it becomes a file, without publishing the idea.

WHAT IT IS

An idea record with a reference, encrypted title and description, privacy level, status, commitment, ledger transaction, revisions, encrypted notes, a reveal action, a certificate, and an evidence package.

WHY IT MATTERS

The most vulnerable moment is before the work exists. Pitching an idea under NDA still leaks it. A commitment lets the creator prove they held the idea at a point in time while keeping the text encrypted.

HOW IT WORKS

- The creator writes a title and description and chooses private, timestamp_only, or organization privacy.
- The description and notes are encrypted with Fernet before storage; the commitment is recorded in the ledger.
- Revisions add encrypted updates without destroying the earlier record.
- Reveal changes the status, records the time, and writes an audit event; the secret is still not returned.
- The certificate and evidence package expose the commitment and timestamp, not the encrypted body.

WHAT YOU SEE

The vault grid shows each idea's commitment, privacy, status, and last update. The detail page shows the ledger reference, revision history, encrypted notes, and reveal state.

LIMITS

An idea commitment proves possession of a committed description, not that the idea is novel, lawful, or commercially usable. Reveal is an intentional disclosure event and cannot be undone.

The three privacy levels serve different social situations. Private keeps the idea to the owner. Timestamp_only allows proof that something was committed without exposing the content. Organization makes the record available inside the organization boundary. The encrypted text is never returned in a public response.

The vault is a good example of the product's separation of concerns: the commitment is public enough to prove, the description is encrypted, and the reveal is a recorded decision rather than a default.

Where to use it

- Date a screenplay treatment before a pitch.
- Commit a product concept before a partner meeting.
- Keep an encrypted research direction with a timestamp.

CHAPTER 13 · EARLY WARNING

Duplicate and similarity detection

Detection is not a court. It is an early-warning system that turns a silent copy into a reviewable alert with references and timestamps.

08
OF 20

FEATURE

Duplicate and similarity detection

Catch an exact copy before upload, and a re-encoded image or close text after registration.

WHAT IT IS
A pre-upload duplicate check by SHA-256 and image dHash-64, plus post-registration alerts for exact duplicates and perceptual or textual similarity.

WHY IT MATTERS
A creator usually discovers copying too late. Checking before upload avoids spending storage on a known duplicate; checking after registration preserves a record even when the claimant proceeds.

HOW IT WORKS
- The browser sends the fingerprint and optional perceptual hash to the duplicate-check endpoint.
- Exact SHA-256 matches return the matching references, timestamps, and whether they belong to the same account or organization.
- After registration, exact matches create a duplicate alert for both owners; image dHash similarity of 0.90 or higher creates an alert for the existing owner.
- Text similarity uses four-word shingles and Jaccard similarity with a 0.72 threshold.
- Alerts move through new, under_review, resolved, or dismissed, with investigation notes.

WHAT YOU SEE
The alerts page shows type, status, matched record, fingerprint, similarity details, and notes. The registration result names the alerts it created.

LIMITS
A similarity score is not proof of copying. Perceptual hashing only applies to images, text extraction depends on the file type, and two independent creators can produce similar work. Alerts require human judgment.

The pre-upload check is advisory. A duplicate never blocks registration, because the same file can legitimately belong to a collaborator or be registered by an organization. The system records the conflict rather than deciding it.

Alerts are paired with in-app notifications so the affected owner sees the finding without polling. The alert status is an investigation state, not a legal status.

METHOD	SIGNAL	THRESHOLD
SHA-256	Exact byte identity	Any match
dHash-64	Perceptual image similarity	0.90 or higher
Text shingles	Four-word overlap, Jaccard	0.72 or higher
Title similarity	Normalized title comparison	Reported with the match

CHAPTER 14 · GOVERNANCE

Dispute resolution workflow

Disputes are where a notary product either becomes useful or becomes dangerous. AUTHENTIQ keeps the human decision explicit and gives the reviewer the evidence.

09
OF 20

FEATURE

Dispute resolution workflow

A structured place for competing claims, encrypted evidence, and a reviewer decision.

WHAT IT IS	A dispute record with claimant, respondent, reason, description, status, priority evidence, encrypted evidence files, reviewer-only resolution, and an aggregated evidence report.
WHY IT MATTERS	A claim of copying needs a process: who is asserting what, which records are involved, what evidence exists, and who is authorized to decide. Without that, alerts become arguments.
HOW IT WORKS	• A signed-in user opens a dispute against a registration; the owner becomes the respondent. • The system finds registrations with the same SHA-256 and reports the earliest as priority evidence. • Parties and reviewers upload encrypted evidence; downloads are access-checked. • A verifier or organization admin moves the status through under_review and evidence_requested. • Resolved or rejected requires a resolution summary and records the reviewer and time. • The report aggregates registrations, proofs, versions, transfers, evidence, ledger audit, and audit-chain state.
WHAT YOU SEE	The disputes list shows status and parties. The detail page shows the claim, the earliest matching registration, evidence, status controls, and a downloadable JSON report.
LIMITS	The platform is not an arbitrator. Priority evidence is a factual ordering, not a legal conclusion. Reviewers can be wrong, and the report is only as good as the evidence attached.

Resolution is deliberately narrow. A party cannot resolve their own dispute. An organization admin can resolve only inside their organization. A verifier can resolve any dispute. The backend enforces all three rules even if a client manipulates the request.

The evidence report is JSON, not a PDF, so it can be attached to another system or inspected programmatically. It includes the audit-chain verification result, which tells a reviewer whether the underlying history is internally consistent.

Where to use it

- Respond to a platform takedown with a dated record and evidence report.
- Document a collaborator disagreement before it escalates.
- Give a neutral reviewer a single package instead of a folder of screenshots.

CHAPTER 15 · OWNERSHIP LIFECYCLE

Two-party ownership transfers

Ownership changes. A product that only records the first owner becomes wrong the moment work is sold, gifted, or reassigned.

10
OF 20

FEATURE

Two-party ownership transfers

Hand over a registration with consent from both sides and a permanent history.

WHAT IT IS	A transfer request with a recipient, state machine, optional message, recipient acceptance or rejection, owner confirmation, cancellation, ledger event, and append-only history.
WHY IT MATTERS	A unilateral edit would destroy the evidence value of the record. A two-party flow proves that both the old and the new owner agreed to the change.
HOW IT WORKS	• The current owner proposes a transfer to an existing account. • The request enters pending_recipient and notifies the recipient. • The recipient accepts or rejects; acceptance moves it to recipient_accepted. • The owner confirms, which changes the registration owner and appends a ledger event. • Either party can cancel while the request is pending or accepted; history is never deleted.
WHAT YOU SEE	The transfers page separates incoming and outgoing requests, shows state, recipient, message, and ledger transaction, and offers only the actions valid for the current state.
LIMITS	The recipient must already have an account. The flow does not move files between storage accounts; it changes the ownership relation. A completed transfer is reversible only by another transfer.

The state machine is small on purpose: pending_recipient, recipient_accepted, rejected, cancelled, completed. Every transition is authorized against the acting party, so a recipient cannot confirm on the owner's behalf and an owner cannot accept on the recipient's behalf.

The history endpoint preserves every transfer for a registration, including rejected and cancelled requests. That history is part of the evidence trail, even when the transfer never completed.

CHAPTER 16 · COLLABORATION

Team projects and unanimous ownership

Creative work is usually collaborative, but ownership records often pretend it is not. Team projects model contributions and ownership without hiding the disagreement.

11
OF 20

FEATURE

Team projects and unanimous ownership

Invite collaborators, approve contributions, and allocate exactly 100 percent with everyone's consent.

WHAT IT IS	A team project with an owner, invited members, roles, contribution records, approval state, ownership allocation proposals, mandatory 100 percent totals, unanimous member approval, a ledger event, and a certificate.
WHY IT MATTERS	A split recorded by one person is not consent. Requiring every active member to approve a 100 percent allocation produces a record that reflects an actual agreement.
HOW IT WORKS	• The creator opens a project and invites members by email with a role. • The invite response returns a one-time invite code and a join link that carries the code. • The invitee signs in, opens Teams, and either accepts the pending invitation or pastes the code into Join with an invitation code. • Redeeming the code activates the membership with the role the owner assigned, and the code is cleared so it cannot be reused. • Members submit contributions, and a manager approves them. • The manager proposes percentages; the backend rejects any total that is not exactly 100. • Every active member approves; the allocation becomes approved only when all have agreed. • The approved allocation is recorded in the ledger and a team certificate can be downloaded.
WHAT YOU SEE	The teams page lists pending invitations and offers a Join with an invitation code field. The team page shows members, roles, contributions, approval state, allocation percentages, and the ownership ledger transaction. The detail page shows the certificate action and your own role.
LIMITS	The project does not escrow money, pay royalties, or enforce an off-platform agreement. It records the digital consensus. Roles are limited to contributor, reviewer, and co_owner. An invite code is a bearer credential, so whoever holds it can take the seat; share it over a private channel.

The allocation check is server-side. A client cannot submit 99 or 101 percent, duplicate a member, or approve on behalf of someone else. The approval loop is deliberately explicit because silent consent is the failure mode this feature exists to prevent.

Team roles and organization roles are different systems. A co_owner on a team project is not an org_admin, and an org_admin is not automatically a team owner.

Because email delivery is not configured, the invite code is the delivery mechanism. That is why the code can be redeemed by any signed-in account and why the seat's email is rewritten to the account that actually joined.

CHAPTER 17 · EVOLUTION

Creative timeline and versions

A single registration captures one moment. A creative project has many: the draft, the client revision, the final master. The timeline preserves that sequence.

12
OF 20

FEATURE

Creative timeline and versions

Keep every important revision, compare fingerprints, and certify the version that shipped.

WHAT IT IS	A searchable, sortable list of registrations plus version records with their own uploads, fingerprints, notes, comparison view, and certificates.
WHY IT MATTERS	Without versions, a creator has to choose which file to register and loses the history of the others. With versions, the evolution itself becomes evidence.
HOW IT WORKS	• A version is created from a completed upload linked to an existing registration. • Each version stores its own fingerprint, file name, size, notes, and timestamp. • The timeline filters by asset type, searches title, reference, fingerprint, and file name, and sorts by date. • The compare view puts two versions side by side with their digests and metadata. • A version certificate can be generated independently of the main registration.
WHAT YOU SEE	The timeline is the main browsing surface. The registration detail page lists versions, and each version has its own download and certificate actions.
LIMITS	Versions are append-only in normal use; deleting a version would weaken the history. A version is not automatically proven; it needs its own share or proof if it is shown to a third party.

The timeline is the closest thing to a library in the product. It is intentionally dense: fingerprint, status, asset type, date, file name, and actions in one row, with search and sorting that work on the server.

Version comparison is a factual display of two records. It does not compute a diff of the content, because the system stores encrypted files and treats byte identity, not textual difference, as the core signal.

CHAPTER 18 · DISCLOSURE

AI provenance declarations

AI provenance is becoming part of ownership. AUTHENTIQ lets the creator make a dated declaration and, when real provider credentials exist, run an analysis rather than inventing a score.

13
OF 20

FEATURE

AI provenance declarations

Record whether a work involved AI, which model or tool, and how much human authorship is claimed.

WHAT IT IS	A provenance record attached to a registration with declared AI involvement, model name, tool or platform, human contribution description, confidence, analysis availability, and a ledger reference.
WHY IT MATTERS	Platforms and clients increasingly ask how a work was made. A dated declaration is more useful than a memory, and honesty about unavailable analysis is better than a fake detector.
HOW IT WORKS	• The creator selects a registration and declares whether AI was involved. • The form records the model, tool, human contribution, and notes. • The declaration is stored with a timestamp and, where applicable, a ledger reference. • Analysis runs only when a real provider is configured; otherwise the UI reports that it is unavailable. • Records are listed per registration and can be added over time as the work evolves.
WHAT YOU SEE	The provenance page lists records with the declared state, model, confidence, and ledger transaction. The analysis action appears only when a provider is configured.
LIMITS	A declaration is self-reported. AI-content classification is not reliable enough to be treated as proof, and this deployment has no provider configured. The feature records what the creator says, not a forensic conclusion.

The product treats provenance as a disclosure layer, not a gate. A registration remains valid whether the creator declares AI involvement or not. The point is to create a dated, attributable statement that can be compared with later claims.

The API returns honest availability flags. If the analysis provider is missing, the response says so; it does not return a fabricated percentage.

CHAPTER 19 · ACCOUNTABILITY

Append-only audit trail

The ledger records ownership events. The audit trail records what people did: sign-ins, registrations, proof attempts, shares, transfers, reviews, and settings changes.

14 OF 20

FEATURE

Append-only audit trail

Every security-relevant action leaves a hash-chained entry that can be verified later.

WHAT IT IS	An append-only audit log with actor, action, entity, metadata, timestamp, previous hash, and record hash, plus an endpoint that recomputes the chain.
WHY IT MATTERS	When something goes wrong, the question is usually who did what and when. A mutable log answers nothing. A hash chain makes silent edits detectable.
HOW IT WORKS	• Security-relevant actions call a shared audit append function inside the same transaction as the action. • Each entry hashes the previous hash plus canonical JSON of the entry body. • Entries are ordered by creation time and id. • Appends are serialized by a reentrant process lock and, on PostgreSQL, a transaction-scoped advisory lock, so two parallel requests cannot read the same tail and fork the chain. • The verification endpoint recomputes the chain and reports the first mismatches. • The interface offers a personal scope and, for reviewers and admins, an all-events scope.
WHAT YOU SEE	The Ledger and audit page has a ledger tab and an audit tab, each with filters, hashes, inspect actions, and a verify button.
LIMITS	Tamper-evident, not tamper-proof. An operator with database access can rewrite the chain. The audit trail is a detection and review aid, not an immutable third-party witness.

Audit entries are written for failed as well as successful actions where relevant. A rejected proof, a failed login, or a blocked upload is evidence of an attempt, not just an error.

The audit chain and the ledger chain are separate. The ledger proves ownership history; the audit chain proves account and operational history. The Integrity Center checks both.

A hash chain only proves order if the append is serialized. Without a lock, two simultaneous requests can both read the same previous hash and produce a fork that verification then flags as tampering. Serializing the append is what keeps the chain meaningful.

CHAPTER 20 · AWARENESS

Notification inbox

A notary product is asynchronous. A transfer arrives, a proof verifies, a dispute opens, an alert fires. The inbox is the human-facing consequence of those events.

15
OF 20

FEATURE

Notification inbox

The product tells you when something needs attention instead of waiting for you to poll.

WHAT IT IS	In-app notifications with category, title, body, read state, related entity, and dismiss action, created in the same transaction as the triggering event.
WHY IT MATTERS	An event that nobody sees is operationally useless. The inbox closes the loop between a backend transition and the person who must act on it.
HOW IT WORKS	• The triggering service writes a notification inside the transaction. • The inbox supports unread-only filtering, category filtering, pagination, mark-one-read, mark-all-read, and dismiss. • The header bell shows the unread count and opens the inbox. • Opening a notification can navigate to the related registration, idea, team, dispute, or transfer.
WHAT YOU SEE	The notifications page groups items by category, shows unread emphasis, and offers an open-related-record action.
LIMITS	Notifications are in-app only. There is no email or push delivery in this deployment, so a user who never signs in will not see them.

The inbox is intentionally not a chat or a comment system. It carries system events with a link to the relevant record. That keeps the audit trail as the source of truth and the inbox as a convenience.

Dismissing a notification removes it from the inbox, not the underlying event. The audit log and ledger remain the durable record.

CHAPTER 21 · SITUATIONAL AWARENESS

Dashboard and activity heatmap

The dashboard is the first screen after sign-in. It should answer three questions in seconds: what do I have, what needs attention, and is the history healthy?

16
OF 20

FEATURE

Dashboard and activity heatmap

A live summary of the workspace, computed from the records you are allowed to see.

WHAT IT IS	Eight stat cards, a 26-week creation cadence heatmap, recent registrations, ledger status, proof statistics, and a recent-activity feed.
WHY IT MATTERS	Creators do not want to browse five pages to learn that a transfer is pending or a proof failed. Aggregation turns scattered records into a working picture.
HOW IT WORKS	• The summary endpoint aggregates counts scoped to the user and organization. • The heatmap covers 182 days and counts registrations, ideas, proofs, and versions per day. • Recent registrations and ledger transactions are limited to a short, readable list. • The security alert count includes new and under-review alerts. • Every card links to the page that can act on it.
WHAT YOU SEE	The dashboard opens with eight cards, then charts and recent activity. The ledger chip in the header shows the active mode and network.
LIMITS	The dashboard is a summary, not an export. Counts depend on access scope, so two users can legitimately see different totals. An empty workspace shows zeros rather than seeded sample data.

The heatmap is a real rendering of daily counts, not a decorative pattern. A new workspace is blank because there is no activity, which is the honest state.

The dashboard refetches on demand. There is no background polling loop, so the refresh button is the explicit way to update a view after an action elsewhere.

CHAPTER 22 · NAVIGATION

Global search

A workspace becomes hard to navigate after a few dozen records. Search is the difference between a demo and a tool.

17 OF 20

FEATURE

Global search

One query across registrations, ideas, teams, disputes, and alerts, scoped to what you can access.

WHAT IT IS	A header search that queries registrations, ideas, teams, disputes, and alerts with a minimum query length of two characters and per-entity result limits.
WHY IT MATTERS	Users remember a title, a reference, or a file name, not which page they used. Search collapses the navigation tree into a single action.
HOW IT WORKS	• The query is normalized and matched with case-insensitive patterns. • Registrations match title, reference, fingerprint, and file name. • Ideas match title and reference; teams match name; disputes match reference and reason; alerts match type. • Results are limited per entity and returned with a total count. • Every result links to the relevant detail page.
WHAT YOU SEE	The header search opens a grouped result panel with entities and counts. An empty result is explicit, not a spinner that never resolves.
LIMITS	Search is substring-based, not semantic. It does not search encrypted file contents, commitment secrets, or comments. Reviewers do not get a global registration search.

Search is deliberately scoped in the backend. A verifier searching for a registration they do not own will not find it, even though they can review disputes. That is a privacy decision, not an oversight.

The minimum length prevents expensive wildcard queries on every keystroke. The client debounces and the API enforces the limit.

CHAPTER 23 · ACCOUNT CONTROL

Settings, sessions, and organization controls

Settings is where identity and security become concrete: who you are, which sessions are open, and what your organization can see.

18
OF 20

FEATURE

Settings, sessions, and organization controls

Change your profile, see your sessions, and manage the organization boundary.

WHAT IT IS

Profile editing, avatar color, role and account metadata, active session list with revocation, organization details, admin provisioning of new members with one-time generated passwords, member password rotation, organization rename for org admins, and a security capability report.

WHY IT MATTERS

Users need a way to recover from a lost device or a suspicious session. Organizations need a canonical name, a member view, and a fast way to give a new colleague working credentials without asking them to self-register. Both need honest reporting of what security features exist.

HOW IT WORKS

- Profile changes update the display name and avatar color and write an audit event.
- The sessions endpoint lists active sessions; deleting one revokes it.
- A password change revokes all sessions and requires the new password on next sign-in.
- The organization endpoint returns members and can_manage; only the owner or an org_admin can rename.
- An admin adds a member with a display name, email, and role. The backend rejects duplicate emails, generates a sixteen-character password with mixed character classes, stores only the Argon2id hash, and returns the plaintext once in the response.
- Member creation writes an audit event and an in-app notification. The one-time credential card can copy both the email and password to the clipboard.
- New password rotates a member credential, clears lockout counters, and revokes every session for that member. The owner's password is protected from admin reset.
- The security endpoint reports Argon2id, token lifetimes, session count, encryption, audit state, and unavailable MFA or passkeys.

WHAT YOU SEE

The settings page has profile, security, organization, and appearance tabs. The organization tab shows the member list with Owner, You, and role badges, a New password action per member, the one-time credential card, and the add-member form. Unavailable capabilities are shown with the reason, not a disabled fake switch.

LIMITS

There is no billing and no MFA or passkey enrollment. Email delivery is not configured, so generated credentials are shown once and must be shared manually; only the Argon2id hash is stored. Provisioning is limited to thirty creations per administrator per hour, and a member cannot provision other members unless promoted to org_admin.

The security report is intentionally blunt. It says passkeys and TOTP are not configured, because a green checkmark for a feature that does not exist would be worse than an honest gap.

Session revocation is server-side. Clearing localStorage in the browser is not enough; the refresh token is invalidated in the database, so a copied token cannot be refreshed after revocation.

Generated passwords are not emailed and cannot be recovered later. That is the point: a password the platform can show again is a password the platform can leak. Lost credentials are replaced by rotation, which also signs the member out everywhere.

The member's workspace identity is real from the first request. They can sign in, open the same organization records, and appear in the organization list immediately. Their role maps to org_member or org_admin at the account layer.

CHAPTER 24 · FOUNDATION

Security controls and rate limiting

This feature is ranked nineteenth because users never see it directly. It is also the reason every other feature can be trusted.

19
OF 20

FEATURE

Security controls and rate limiting

The quiet layer: password hashing, token rotation, lockout, limits, and backend authorization.

WHAT IT IS

Argon2id password hashing, JWT access and refresh tokens, hashed refresh storage, session revocation, login lockout, per-scope rate limits, CORS, security headers, encrypted storage, and backend authorization checks.

WHY IT MATTERS

A beautifully designed proof system is worthless if anyone can read another user's records or replay a token. Security is the substrate, not a feature card.

HOW IT WORKS

- Passwords are hashed with Argon2id; plaintext is never stored.
- Access tokens last 45 minutes; refresh tokens last 14 days, are stored hashed, and rotate on use.
- Login allows eight attempts per IP and email in five minutes, then locks the account temporarily.
- Registration allows ten attempts per IP per hour; proof generation forty per user per hour; share creation thirty per user per hour; public verification thirty per IP per ten minutes.
- Every protected route loads the user from the database and checks ownership, organization, or reviewer scope.

WHAT YOU SEE

The settings security section and the API error responses make the controls visible. Rate-limit responses include Retry-After.

LIMITS

No MFA, passkeys, email verification, or hardware key custody. Rate limiting is in-process, so multiple workers need shared infrastructure. TLS termination is a deployment responsibility.

The refresh flow is single-flight in the browser: concurrent 401 responses share one refresh request, then retry once. If refresh fails, local credentials are cleared and the user is returned to sign-in with the attempted path.

CORS is restricted to the configured origins, and the API sets security headers. Neither replaces a proper reverse proxy and TLS configuration in production.

SCOPE	LIMIT	WINDOW
Login	8 per IP and email	5 minutes
Registration	10 per IP	1 hour
Proof generation	40 per user	1 hour
Share creation	30 per user	1 hour
Public verification	30 per IP	10 minutes

CHAPTER 25 · DELIVERY

Operations, documentation, and tests

The last feature is the ability to run, inspect, and trust the system. It is unglamorous and it is what separates a prototype from a demo.

20
OF 20

FEATURE

Operations, documentation, and tests

Health checks, OpenAPI, Docker, a 41-test suite, and documentation that matches the code.

WHAT IT IS	A health endpoint, OpenAPI docs, a Docker Compose topology, an Nginx SPA server, pytest coverage, a TypeScript build, a seed command, and six documentation files.
WHY IT MATTERS	If you cannot run the product, inspect its API, and reproduce its claims, you cannot rely on it. Documentation that drifts from code is worse than no documentation.
HOW IT WORKS	• GET /api/health reports status, version, ledger mode, and proof scheme. • OpenAPI is generated at /api/docs and /api/openapi.json. • Docker Compose runs PostgreSQL, the backend, and an Nginx-served frontend with health checks and volumes. • The pytest suite covers authentication, uploads, registration, ZKP, ledger, audit, teams, transfers, disputes, and dashboard aggregation. • The seed command creates three demo logins and nothing else.
WHAT YOU SEE	The health payload, the Swagger UI, the Compose topology, and the test output are all inspectable. The docs folder covers architecture, stack, API, security, ZKP, and the optional chain.
LIMITS	No background workers, no production migration history, no email delivery, and no on-chain proof verification. Docker and Hardhat assets exist but were not executed in this local verification.

The test suite is the strongest evidence that the claims are real. It tampers with proofs, ledgers, and audit chains and expects failure. A mock would pass; these tests fail when the cryptography is wrong.

The documentation set is part of the product: README for orientation, ARCHITECTURE for flows, TECH_STACK for versions, API for endpoints, SECURITY for controls, and ZKP for the exact proof statement.

PART III

PAGE BY PAGE

Every public route and every protected workspace page, in the order a real session encounters them. Use this part as a map when you are testing the product or training someone else.

This part describes what appears on screen and which action belongs there. It does not repeat the technical mechanics from Part II unless the page depends on them.

IN THIS PART

26 Public pages

27 Workspace pages

PART III OF V

CHAPTER 26 · PAGE GUIDE

Public pages

Eight routes are open to anyone: the landing page, sign-in, sign-up, password reset, public verification, the public records registry, the organization join page, and the not-found page.

ROUTE	PURPOSE	AUTHENTICATION
/	Marketing site and interactive product preview.	None
/login	Sign in, use a demo account, or start password recovery.	None
/signup	Create an individual or organization account.	None
/reset-password	Consume a reset token and choose a new password.	None
/verify/:verificationId	Disclosure-limited public verification and proof check.	None
/verify	Redirect to the signed-in verification page.	None, but the target is protected
/records	Public records registry of records whose owner chose the Public disclosure level.	None

ROUTE	PURPOSE	AUTHENTICATION
/join	Redeem an organization invitation code and join that organization.	None to view, sign-in to accept
*	Not-found page with routes back to safety.	None

Landing page · /

The landing page is a long editorial narrative, not a dashboard preview. It opens with the core promise and a primary call to action, then walks through the problem, the four technologies, the workflow, the applications, and the FAQ. The navigation links to the same anchors and to sign-in and sign-up.

- Hero and navigation. The AUTHENTIQ wordmark, product promise, and calls to action. The mobile menu collapses the anchors.
- Technology cards. SHA-256 fingerprinting, zero-knowledge proofs, blockchain ledger, and dispute resolution, each with a small explanatory diagram.
- Workspace preview. A simulated workspace that hashes a selected file with the real Web Crypto API. The proof and verification panels are illustrative and are not connected to the API; the page does not claim they are.
- Applications. Art and design, photography and media, software and source code, research and documentation, and AI-generated content.
- FAQ and legal modals. Plain-language answers, terms of use, and a privacy policy that names the limitations.
- Motion and accessibility. The liquid cursor is disabled for reduced-motion users and touch devices; focus states use the lime accent.

Sign in · /login

A focused card with email, password, show or hide control, and a primary sign-in button. Three demo account shortcuts fill the form without submitting. A forgot-password dialog requests an email and displays the development reset token when email delivery is unavailable.

SUCCESS	Tokens and user profile are stored, the refresh timer starts, and the user is sent to the attempted page or /dashboard.
FAILURE	A clear error is shown. Repeated failures lock the account; the API returns 423 or 429.
RESTORATION	If a session already exists, /api/auth/me runs first and the page redirects to the workspace.
DEMO	demo@authentiq.dev, verifier@authentiq.dev, maya@authentiq.dev. The cards expose the passwords for evaluation.

Create an account · /signup

The form switches between individual and organization. Both ask for display name, email, password, and confirmation. Organization adds a name field and explains that the creator becomes the organization admin. The password minimum is ten characters; the client validates before the request and the server enforces the same rule.

- On success, the session is created immediately and the user lands in the workspace.

- The new account starts empty: no registrations, ideas, teams, alerts, transfers, disputes, or ledger events.
- Organization sign-up creates the organization, the owner membership, and the org_admin role in the same flow.
- The legal copy is available from the form; acceptance is part of account creation.

Reset password · /reset-password

The page accepts a reset token and a new password. In this deployment there is no email provider, so the token is returned in the forgot-password response and can be pasted here. The token lasts 30 minutes. A successful reset revokes every existing session, which is the correct behavior after a suspected compromise.

Public verification · /verify/:verificationId

This is the page a client or platform sees. It does not require an account and never serves the original file. It loads the disclosure-approved metadata, then optionally accepts a proof package pasted into a text area or supplied by the share flow.

ELEMENT	WHEN IT APPEARS	WHY
Title	Only for a public record	Titles can reveal unreleased work.
Fingerprint	Only when the share enabled it	A fingerprint can be correlated with other databases.
Commitment	Unless the record is private	The commitment is the public half of the ownership claim.
Ledger transaction	When a record exists	Shows the ordered history reference.
Proof result	After a package is submitted	Returns valid or invalid with a reason.
Owner identity	Never	The public endpoint does not expose personal identity.

THE PUBLIC PAGE IS INTENTIONALLY BORING

It shows a small set of facts, a proof result, and a clear privacy note. It is not a second workspace, and it cannot be used to enumerate records or accounts.

Public records · /records

A public registry of the records whose owners deliberately chose the Public disclosure level. It needs no account, and it lists only those records. Search, asset-type filtering, and sorting are server-side. Private records and fingerprint-only records are never enumerated here; they stay reachable only through their reference or a silent-sharing link.

- Each entry shows the title, creator display name, organization or team affiliation, asset type, registration date, and the public fingerprint.
- Descriptions are truncated to a preview length so the registry cannot be scraped for full text.
- Searching returns only public records. The query is applied inside the disclosure filter, not after it.

Join an organization · /join

The redemption page for an organization invitation. The admin issues a one-time code and shares the link that carries it. The recipient opens the page, sees which organization and role the code grants, signs in if needed, and accepts. Accepting creates the membership and the org_member role in one transaction, and the code is cleared so it cannot be reused.

Not found · *

An unknown path renders a calm not-found page with a route back to the landing page and, when signed in, the dashboard. It does not leak whether a protected resource exists.

CHAPTER 27 · PAGE GUIDE

Workspace pages

Nineteen protected pages share one shell: a grouped sidebar, global search, notifications, a ledger status chip, and a user menu. Every page is reachable from the sidebar, but the backend still enforces access.

The application shell

WORKSPACE GROUP	Dashboard, Register creation, Creative timeline, Idea vault, AI provenance.
COLLABORATION GROUP	Teams, Transfers, Disputes.
TRUST CENTER GROUP	Verify ownership, Integrity center, Alerts, Ledger and audit, Notifications, Settings.
HEADER	Global search with grouped results; notification bell with unread count; ledger mode chip; user menu.
MOBILE	The sidebar becomes a drawer. The header keeps search and the account menu.

ROUTE	PAGE	PRIMARY JOB
/dashboard	Dashboard	Summarize counts, activity, ledger state, and pending work.
/register	Register creation	Upload, verify, and register a new work.
/timeline	Creative timeline	Browse, filter, search, and sort registrations.
/registrations/:id	Registration detail	Inspect a record and download certificate, file, or packet.
/ideas	Idea vault	Protect ideas before a file exists.
/ideas/:ideaRef	Idea detail	Read, revise, reveal, and certify one idea.
/provenance	AI provenance	Declare AI involvement and inspect analysis availability.
/teams	Teams	Create, join, and open team projects.
/teams/:id	Team detail	Members, contributions, allocation, certificate.
/transfers	Transfers	Propose, accept, reject, cancel, or confirm a transfer.
/disputes	Disputes	Open and filter disputes.

ROUTE	PAGE	PRIMARY JOB
/disputes/:id	Dispute detail	Read the claim, attach evidence, resolve if authorized.
/anonymous-verification	Verify ownership	Generate a challenge, proof, or share link.
/alerts	Alerts	Investigate duplicate and similarity findings.
/notifications	Notifications	Read, filter, open, and dismiss system events.
/audit-logs	Ledger and audit	Inspect transactions and audit entries, run verification.
/integrity	Integrity center	Recompute chains and workspace integrity.
/settings	Settings	Profile, organization, sessions, and security report.
/organization	Organization workspace	Shared organization records, members, and invitations.
/admin	Site administration	Platform users, organizations, disputes, and audit trail.

Dashboard · /dashboard

The first page after sign-in. Eight cards show total creations, verified records, protected ideas, team projects, pending transfers, active disputes, security alerts, and verified proofs. Below them sit a 26-week activity heatmap, recent registrations, and a ledger status panel.

- What you can do: jump to any card's page, refresh the summary, or open a recent registration.
- Empty state: zeros and a clear prompt to register the first work. No sample records are invented.
- What it reads: /api/dashboard/summary and /api/dashboard/activity, scoped to your access.

Register creation · /register

A single guided form. Choose a file, watch the local SHA-256, wait for the duplicate check, then supply title, asset type, description, tags, privacy mode, and an optional creation date. The upload runs in chunks with a visible digest, and the result card shows the fingerprint, both commitments, the ledger transaction, privacy, and the number of alerts created.

- Before upload: extension and size validation, local hash, duplicate and perceptual check.
- During upload: per-chunk progress and verification. A failed chunk can be retried without restarting the file.
- After registration: download the certificate, open the record, or start another registration.
- Failure states: unsupported extension, oversized file, hash mismatch, scanner rejection, or a server error. Each is shown as a specific error, not a generic success.

Creative timeline · /timeline

The library view. Each card shows the title, asset type, status, registration reference, fingerprint, file name, and date. Search matches title, reference, fingerprint, or file name. Filters cover asset type and sort order, and pagination is server-side.

Registration detail · /registrations/:id

The evidence workbench. It displays the registration reference, SHA-256, Poseidon commitment, Pedersen commitment, ledger transaction, file metadata, privacy, status, registered date, and creation date. Actions include preview, download original, download certificate, export evidence packet, and refresh.

- The trust card explains the 0-100 score factor by factor, including penalties.
- The Ownership DNA panel renders the fingerprint as a deterministic grid and offers an SVG download.
- Version history, proof history, alerts, and ledger linkage are visible where applicable.
- Access is limited to the owner, the organization, or an authorized reviewer scope.

Idea vault · /ideas

A grid of protected ideas. Each card shows the title, commitment, privacy, status, and update time. Creating an idea requires a title and a description of at least ten characters, plus a privacy level: private, timestamp_only, or organization.

Idea detail · /ideas/:ideaRef

The full idea record: reference, commitment, ledger transaction, privacy, status, revisions, encrypted notes, and reveal state. Actions add a revision, update notes, reveal the idea, download the certificate, or export the evidence package. Reveal is deliberate and recorded.

AI provenance · /provenance

A list of provenance declarations with model, tool, declared AI state, confidence, and ledger reference. The form records AI involvement, model name, tool or platform, a prompt commitment (the raw prompt is not stored by that field), human contribution, editing details, and notes. Analysis only appears when a real provider is configured.

Teams · /teams and /teams/:id

The list shows projects you own or have joined, with status and an ownership ledger transaction. A Pending invitations card lists projects that invited your email address, each with an Accept invitation button. Below it, Join with an invitation code is a single field: paste the code the team owner shared and your membership activates with the role the owner assigned, then you land on the project. The detail page shows members, roles, contributions, allocation state, your role, and the certificate action. Managers can invite, approve contributions, and propose percentages; every active member must approve a 100 percent allocation. An invite code is single-use and is cleared once redeemed.

Transfers · /transfers

Two lists: incoming and outgoing. Each request shows the registration, the other party, the current state, the message, the ledger transaction, and only the actions valid for that state. Creating a transfer requires an owned registration and the recipient's email address; the recipient must already have an account.

Disputes · /disputes and /disputes/:id

The list shows disputes you are a party to or are authorized to review, with status and filters. Raising a dispute requires a registration, a reason, and a description of at least twenty characters. The detail page shows the parties, the earliest matching registration, encrypted evidence, status controls, and a downloadable JSON evidence report.

Verify ownership · /anonymous-verification

The proof workbench. Select a registration, request a challenge, choose the original file, and watch the local hash compare with the registered fingerprint. When they match, generate and verify the proof. The same page creates a silent-sharing link with an expiry and use limit and displays the proof package.

- Challenge nonce and expiry are shown explicitly.
- A mismatch blocks proof generation and explains why.
- The result includes the validity message, public signals, and recent proofs.
- A share link is displayed once and can be copied; existing links can be listed and revoked.

Alerts · /alerts

An investigation queue. Each alert shows its type, status, matched record, fingerprint, similarity data, and notes. Actions move it to under_review, resolved, or dismissed and record investigation notes. Alerts are findings, not verdicts.

Notifications · /notifications

The inbox. Filter by category or unread state, mark one or all as read, open the related record, or dismiss an item. Notifications are created inside the same transaction as the event that triggered them.

Ledger and audit · /audit-logs

Two tabs. The ledger tab lists transactions with block number, type, actor, entity, record hash, and timestamp, and offers inspection. The audit tab lists security events with actor, action, entity, and record hash. Buttons verify the ledger and the audit chain and report the first mismatch if any.

Integrity center · /integrity

The diagnostic page. Four headline tiles show ledger chain, audit chain, workspace digest, and open issues. Below them are the last hashes, storage presence, ledger linkage, duplicate groups, exposure counts, and a record-level map. Recheck recomputes everything from the source.

Settings · /settings

Profile editing, account metadata, organization details and members, active sessions, and a security report. Display name and avatar color are editable. Sessions can be revoked individually. The security section reports Argon2id, token lifetimes, encryption, audit state, and the honest unavailability of MFA and passkeys.

Organization admins and owners get a provisioning panel. Add a member with a display name, email address, and role of Member or Administrator. The backend creates the account immediately, generates a sixteen-character password that mixes lowercase, uppercase, digits, and symbols, and returns it once in a one-time sign-in card. Copy sends both the email and password to the clipboard. The plaintext password is never stored and cannot be shown again. Email delivery is not configured, so the admin shares the credentials directly and should use a secure channel.

Each row in the member list carries New password. Rotating a member password generates a fresh one-time credential, clears failed-login counters, and revokes every active session for that member. The owner's own password is excluded from admin reset. Every provisioning and rotation action is written to the audit chain and triggers a notification for the affected member.

Organization workspace · /organization

The shared organization view. It lists the organization's registrations with their proof and certificate state, the member roster with roles and statuses, and the outstanding invitations. A manager can issue an invitation, copy the one-time code, or revoke an invitation that was never accepted.

Site administration · /admin

The operator console, gated on the site_admin role at the API rather than in the menu. Tabs cover platform overview counts, user accounts and their status, organizations and their members, disputes awaiting a decision, registrations across the platform, and the audit trail. Actions are deliberately narrow: suspend or reactivate an account, resolve a dispute, and act on an organization. Every write is audited with the operator identity.

A PAGE IS NOT A PERMISSION

The sidebar hides routes a user cannot use, but the real check is in the API. Opening /audit-logs without a token redirects to /login; calling the endpoint without a valid token returns 401, and with an insufficient role returns 403.

PART IV

TECHNOLOGY A TO Z

Every layer from the browser to the database, including the parts a user never sees: hashing, commitments, proofs, encryption, the ledger adapter, PDF generation, storage, and the documentation set.

Version numbers come from the manifests and documentation in this repository. Where the local environment and the container differ, both are named. Anything optional is labelled optional.

IN THIS PART

28 Architecture at a glance
29 Frontend stack A to Z
30 Frontend storage and browser state
31 Backend stack A to Z
32 Data layer and schema
33 Cryptography
34 Blockchain and ledger tooling
35 Files, PDFs, and QR codes
36 Infrastructure, documentation, and configuration

PART IV OF V

CHAPTER 28 · TECHNOLOGY

Architecture at a glance

AUTHENTIQ is a three-tier application with a cryptography-heavy backend. The browser renders and hashes; the API authorizes, proves, encrypts, and records; PostgreSQL and local storage persist.

The tiers

TIER	TECHNOLOGY	RESPONSIBILITY	TRUST POSTURE
Browser	React 18, TypeScript, Vite, TanStack Query	Render the interface, hash files locally, upload chunks, hold the session in localStorage.	Untrusted. Every claim is rechecked by the server.
API	FastAPI, Pydantic, SQLAlchemy, Uvicorn	Authenticate, authorize, validate, hash, prove, encrypt, certify, audit.	Trusted application boundary.
Data	PostgreSQL (Neon) for state; local disk for files, evidence, and the ledger	Relational records, encrypted file containers, append-only history.	Trusted infrastructure. Operator access is the residual risk.

A request from click to record

01 The browser sends a JSON or multipart request with a bearer access token when the route is protected.

02 FastAPI validates the payload, resolves the database session, and loads the current user and session from the database.

03 The route calls a service or performs the domain operation. Authorization is checked against the resource, not only the route.

04 Domain code hashes, encrypts, proves, or writes an audit entry. Cryptography runs in-process; there is no external proving service.

05 SQLAlchemy commits relational state. File writes happen in the storage layer. The ledger append calls fsync.

06 A response is shaped by a serializer. Errors become the documented 4xx or 5xx codes; unhandled exceptions become a generic 500.

Repository map

PATH	WHAT LIVES THERE
backend/app/api/	HTTP routers by domain: auth, uploads, registrations, zkp, sharing, ideas, versions, provenance, teams, transfers, disputes, alerts, ledger, integrity, notifications, audit, search, settings, dashboard.
backend/app/services/	Registration, access control, analysis, certificates, evidence and trust, notifications, serializers, integrity, audit.
backend/app/zkp/ and app/fingerprint/	Poseidon, BN254, Schnorr, SHA-256, chunk hashing, canonical JSON.
backend/app/security/	Argon2id, JWT, encryption, rate limiting, file scanning.
backend/app/storage/ and app/ledger/	AQVL1 encrypted containers and the simulation or Polygon ledger adapters.
backend/app/models/ and app/schemas/	29 SQLAlchemy tables and Pydantic request and response contracts.
frontend/src/pages/	The landing page and every workspace screen.
frontend/src/components/	Shell, brand, common states, UI primitives, and domain widgets.
frontend/src/lib/	The single API client, upload client, formatting, and helpers.
frontend/src/styles/	The house stylesheet and the application theme.
blockchain/ and circuits/	Optional Solidity registry, Hardhat project, and Circom ownership circuit.
docs/ and storage/	Six documents and the runtime file store.

Trust boundaries

BROWSER TO API	TLS in a real deployment. The API never trusts the browser's hash, role, ownership, or proof claims.
API TO DATABASE	A single trusted connection string from the environment. The database is not exposed to the browser.
API TO FILESYSTEM	Encrypted containers for completed files; temporary plaintext chunks exist briefly during upload.
API TO LEDGER	A local adapter with fsync, or a Polygon adapter that fails closed if configuration is missing.
OPERATOR	Can read the database and filesystem. Can rewrite the local ledger and audit chain. Proper deployment requires a secret manager, encrypted volumes, and off-box retention.

Deployment topologies

MODE	TOPOLOGY	USE
Local development	Vite :5173 to FastAPI :8000 to SQLite or Neon plus local storage and JSONL ledger.	Fast iteration, deterministic tests, zero external dependencies.
Hosted state	The current workspace: FastAPI to Neon PostgreSQL, with files and ledger on local disk.	A shared relational database without moving encrypted files off the machine.
Docker Compose	Nginx :8080 serves the SPA and proxies /api to FastAPI :8000, which uses PostgreSQL :5432 and a /data volume.	A reproducible, production-like topology for evaluation.

THE IMPORTANT SPLIT

Relational state lives in PostgreSQL. Encrypted files, evidence, and the simulation ledger live on local disk. Back up both, and do not assume that a database backup contains the files or the ledger.

CHAPTER 29 · TECHNOLOGY

Frontend stack A to Z

The frontend is a React single-page application built with Vite and TypeScript. It owns rendering, local hashing, chunked uploads, and session display; it never owns authorization.

PACKAGE	VERSION	WHERE IT IS USED	WHY IT IS THERE
React	18.3	Every component and page.	Component model and UI runtime.
React DOM	18.3	Browser rendering.	Mounts the SPA.
TypeScript	5.7	All source files.	Static typing and build-time checking.
Vite	6.0	Development server and production build.	Fast startup, static output, /api proxy.
@vitejs/plugin-react	4.x	vite.config.ts.	React fast refresh and JSX transforms.
React Router	6.28	App.tsx and the shell.	Public and protected route trees, redirects, location state.
TanStack Query	5.62	Every data screen.	Server-state caching, mutations, invalidation, retries.
React Hook Form	7.54	Sign-up, registration, idea, team, dispute, and settings forms.	Form state with minimal re-renders.
Zod	3.24	Form schemas and validation messages.	Typed validation close to the form.
@hookform/resolvers	3.9	Form integration.	Connects Zod schemas to Hook Form.
Tailwind CSS	3.4	All styling.	Utility classes and semantic tokens.
Radix UI	1.x and 2.x	Dialog, dropdown, label, tabs, slot.	Accessible primitives without a heavy component framework.

PACKAGE	VERSION	WHERE IT IS USED	WHY IT IS THERE
class-variance-authority	0.7	Button and UI variants.	Typed variant definitions.
tailwind-merge	2.6	The cn() helper.	Conflict-free class composition.
Lucide React	0.468	Icons across the shell and pages.	Consistent stroke icon set.
Recharts	2.15	Dashboard charts and heatmap support.	Composable SVG charts.
Framer Motion	11.15	Page and component transitions.	Declarative motion with reduced-motion support.
Sonner	1.7	Toast feedback.	Short, accessible success and error messages.
date-fns	4.1	Date formatting and relative time.	Small, tree-shakeable date utilities.
Funnel Display	woff2	Body and interface copy.	Self-hosted brand type, no external font request.
Turret Road	woff2	Display headings.	Self-hosted geometric display type.

The design system in code

TOKEN	VALUE	WHERE IT APPEARS
--paper	#FFFFFF	Page and card background.
--light	#EAEAEA	Secondary surfaces, table alternation, callouts.
--dark	#171717	Ink, primary text, borders, header bars.
--lime	#B7FF00	Accent, focus ring, highlights, chart emphasis.
--muted-foreground	#888888	Secondary text, labels, inactive states.
--border	#D0D0D0	Card edges and table rules.
--warning	#454545	Dark grey caution state.

The landing stylesheet is loaded before the application stylesheet, so the palette, type faces, and base element rules apply to the workspace too. The Tailwind theme maps semantic tokens onto the same values and remaps the legacy blue, indigo, violet, emerald, and slate scales onto ink, lime, and neutral greys. There is one theme and no theme switch, which prevents surface drift.

Build and runtime conventions

- One HTTP entry point. src/lib/api.ts adds the bearer token, performs a single-flight refresh on 401, retries once, extracts errors, and handles authenticated blob downloads.
- One upload client. src/lib/upload.ts slices the file, hashes each chunk, sends it, and tracks progress.
- Session restoration. AuthContext validates the stored token with /api/auth/me; the cached profile is a display shortcut.
- Protected routing. ProtectedShell redirects unauthenticated users to /login and stores the attempted path in location state.

- Build. npm run build runs tsc --noEmit and then vite build. The output is a static bundle that Nginx can serve with history fallback.

WHAT THE BROWSER DOES NOT DECIDE

The client can hide a button, but it cannot grant a role, approve a proof, or read a registration the API refuses to return. Route visibility is user experience; backend authorization is the boundary.

CHAPTER 30 · TECHNOLOGY

Frontend storage and browser state

The browser stores exactly three values for authentication, keeps server data in memory, and uses short-lived blob URLs for protected downloads. There is no IndexedDB, service worker cache, or cookie-based session.

What is stored where

MECHANISM	KEY OR USE	LIFETIME	WHY
localStorage	authentiq_access_token	Until logout or expiry	Bearer token for API requests.
localStorage	authentiq_refresh_token	14 days or until revoked	Rotates the access token after a 401.
localStorage	authentiq_user	Until logout	Cached profile for fast rendering; not an authorization source.
TanStack Query cache	Server responses by query key	In memory until reload or invalidation	Avoids duplicate fetches and keeps screens fast.
Blob URLs	Authenticated file and certificate downloads	Until revoked after the click	Lets the browser download a protected response without a public URL.
File API	The selected file	Until the upload finishes	Chunk slicing and local SHA-256.
Web Crypto	SHA-256 digest	Request-scoped	Client-side fingerprint feedback and duplicate check.

What is deliberately not stored

- No session cookie. Authentication uses explicit bearer headers.
- No IndexedDB or offline database. The workspace requires the API.
- No service worker cache. A stale bundle cannot quietly serve old authorization logic.
- No commitment secret, file key, or raw password. Those never reach the browser.
- No analytics identifier or third-party script. Fonts are self-hosted.

The refresh flow

When a protected request returns 401, the API client starts one refresh request and shares it across concurrent failures. On success it replaces both tokens and the cached user, then retries the original request once. On failure it clears all three localStorage keys and sends the user to /login. That single-flight behavior prevents a burst of parallel refreshes from invalidating its own rotation chain.

Download handling

Protected files, certificates, and evidence packets are fetched with the bearer token, converted to a Blob, and exposed through a temporary object URL. The URL is revoked after the download is triggered. A user cannot copy a durable public file URL out of the interface because one does not exist.

Upload handling

The file is read in slices of at most 8 MB. Each slice is hashed with Web Crypto and sent as multipart form data. The browser keeps only the current slice and the accumulated progress in memory. The overall digest is declared at completion and recomputed by the server. This keeps memory bounded even for a 200 MB file.

THE TRADE-OFF TO UNDERSTAND

Storing tokens in localStorage is simple and works with a static SPA, but it means an XSS bug could read them. The mitigation is a strict content policy, no third-party scripts, React escaping, and short access-token lifetimes. A production system with a same-site backend should consider an HttpOnly cookie with CSRF protection.

Clearing state

LOGOUT	Calls /api/auth/logout, revokes the refresh token, removes all three keys, clears the query cache, and navigates to /login.
REFRESH FAILURE	Removes local credentials and returns to sign-in with the attempted path.
PASSWORD CHANGE	The server revokes every session; the current client is signed out on the next refresh.
SESSION REVOCATION	A revoked session cannot refresh even if the browser still holds the old token.

CHAPTER 31 · TECHNOLOGY

Backend stack A to Z

The backend is a synchronous FastAPI application. It validates, authorizes, hashes, proves, encrypts, certifies, and records. PostgreSQL holds state; local disk holds encrypted files and the ledger.

PACKAGE	VERSION	WHERE IT IS USED	WHY
Python	3.12 container, 3.14 local	Runtime.	Modern typing and performance.
FastAPI	0.115+	HTTP routing and dependency injection.	Typed request models, OpenAPI generation.
Uvicorn	0.34+	ASGI server.	Simple, fast application server.

PACKAGE	VERSION	WHERE IT IS USED	WHY
Pydantic	2.10+	Request and response schemas.	Strict validation and serialization.
pydantic-settings	2.7+	Environment configuration.	AUTHENTIQ_ prefixed settings with validation.
SQLAlchemy	2.0+	ORM and sessions.	Explicit models, relationship loading, portable SQL.
psycopg	3.2+	PostgreSQL driver.	Neon and Compose PostgreSQL connectivity.
python-jose	3.3+	JWT access and refresh tokens.	Claims, expiry, signing.
argon2-cffi	23.1+	Password hashing.	Argon2id with per-password salts.
cryptography	44+	AES-GCM and Fernet.	File, evidence, idea, and secret encryption.
ReportLab	4.2+	PDF certificates and reports.	Programmatic, deterministic PDFs.
qrcode	8.0+	Certificate QR codes.	Offline link to the verification page.
Pillow	11+	QR image handling.	Rendering and image metadata support.
httpx	0.28+	Outbound HTTP and Polygon JSON-RPC.	Timeouts and a single client API.
python-multipart	0.0.20+	Chunk and evidence uploads.	Multipart parsing in FastAPI.
email-validator	2.2+	Email normalization.	Rejects malformed addresses.
pytest	8.3+	Backend test suite.	76 integration and security tests.
pytest-asyncio	0.25+	Async test support.	Exercises async code paths.
Alembic	listed	Schema migrations.	Dependency is present; no revision history yet.

Layering

LAYER	LOCATION	RESPONSIBILITY
HTTP	app/api/	Routing, status codes, payload validation, response shaping.
Dependencies	app/core/deps.py	Database session, client IP, current user, token validation.
Services	app/services/	Registration, access, analysis, certificates, evidence, integrity, notifications, audit, serializers.
Domain crypto	app/zkp/, app/fingerprint/, app/security/	Poseidon, BN254, Schnorr, SHA-256, passwords, JWTs, encryption.
Storage	app/storage/	AQVL1 containers, chunk handling, extension and malware gates.
Ledger	app/ledger/	Simulation and Polygon adapters behind one interface.
Models and schemas	app/models/, app/schemas/	SQLAlchemy tables and Pydantic contracts.
Configuration	app/core/config.py	Environment settings and path resolution.

Why synchronous handlers

Hashing, encryption, proof generation, and certificate rendering are CPU- and I/O-local tasks. Synchronous handlers keep the flow readable and deterministic, and the workload is not highly concurrent. A production deployment that expects sustained load should move certificate and report generation to a worker queue and run multiple API workers behind a shared rate limiter.

Error model

STATUS	MEANING	TYPICAL CAUSE
400	Invalid state or bad request	Malformed transition or missing prerequisite.
401	Missing, expired, or invalid token	No bearer token, expired access token.
403	Authenticated but not authorized	Wrong owner, organization, or role.
404	Not found or hidden by access control	A record the caller may not see.
409	State conflict or replay	Usage limit reached, duplicate transition.
410	Expired or revoked	Share link expired, revoked, or consumed.
413 / 415	Too large or unsupported type	File over 200 MB or extension outside the list.
422	Validation or integrity failure	Chunk hash mismatch, proof mismatch.
423	Account locked	Too many failed logins.
429	Rate limit exceeded	Inspect Retry-After.
500	Unexpected server error	Details logged server-side, not returned.

ONE BACKEND RULE

Pydantic validates shape. Services validate state. Access helpers validate relationships. None of them trusts a role or owner id supplied by the client.

CHAPTER 32 · TECHNOLOGY

Data layer and schema

Relational state lives in PostgreSQL, currently a hosted Neon database. SQLite remains the zero-setup fallback and the test database. Files and the ledger are not in the database.

The active configuration

DATABASE	PostgreSQL on Neon, pooled connection with SSL required.
DRIVER	psycopg 3 through SQLAlchemy 2.
ENGINE OPTIONS	pool_pre_ping=True, pool_recycle=280 seconds, connect_timeout=15 seconds.
SCHEMA CREATION	SQLAlchemy metadata at application startup. No migration revisions are stored yet.
LOCAL FALLBACK	sqlite:///backend/authentiq.db when AUTHENTIQ_DATABASE_URL is unset.
TEST DATABASE	An isolated SQLite database created by the pytest fixtures.

Twenty-nine tables in five domains

DOMAIN	TABLES	COUNT
Identity	users, organizations, organization_members, user_sessions, password_reset_tokens	5
Registrations	registrations, file_metadata, uploads, file_chunks, commitments, zkp_challenges, zkp_proofs, verification_requests, ledger_transactions	9
Content	ideas, idea_revisions, creative_versions, ai_provenance	4
Collaboration	team_projects, team_members, contribution_records, ownership_allocations	4
Governance and operations	ownership_transfers, theft_alerts, similarity_analyses, disputes, dispute_evidence, notifications, audit_logs	7

What lives where

DATA	LOCATION	BACKUP IMPLICATION
Users, registrations, metadata, proofs, teams, transfers, disputes, notifications, audit entries	PostgreSQL	A database backup or provider point-in-time recovery.
Encrypted original files and versions	storage/uploads on local disk	Must be backed up separately; the database only stores the path.
Encrypted dispute evidence	storage/evidence on local disk	Must be backed up with the disputes they support.
Simulation ledger	storage/ledger.jsonl	Append-only file; back it up with the database to preserve linkage.
Certificates	Generated on demand	Nothing to back up; regenerate from the record.

Connection and portability

The engine is created from AUTHENTIQ_DATABASE_URL. PostgreSQL URLs are normalized to the psycopg driver. The pool is configured to survive a hosted database that closes idle connections: pre-ping checks a connection before use, recycle retires it before the provider timeout, and a 15-second connect timeout prevents a hung request. SQLite is kept for tests because it is fast, isolated, and requires no service.

Schema lifecycle

Tables are created from SQLAlchemy metadata at startup. That is convenient for a prototype and dangerous for production: it does not version changes, backfill data, or coordinate zero-downtime migrations. Alembic is listed as a dependency but no revision history is included. A real deployment should generate an initial migration, switch startup to check-only, and run migrations as a separate step.

THE SPLIT THAT SURPRISES PEOPLE

A Neon backup restores users, registrations, and metadata. It does not restore encrypted files or ledger.jsonl. If the filesystem is lost, the records survive but the original bytes and the local chain do not.

CHAPTER 33 · TECHNOLOGY

Cryptography

The cryptographic layer is small, explicit, and implemented in Python. It separates content hashing, ownership commitments, zero-knowledge proofs, password hashing, token signing, and symmetric encryption.

PRIMITIVE	IMPLEMENTATION	PURPOSE	WHERE
SHA-256	Python hashlib and browser Web Crypto	Canonical file, chunk, record, and audit hashes.	Uploads, fingerprint API, ledger, audit, certificates.
Poseidon-254	Pure Python with circomlib-compatible BN254 constants	Commitment over fingerprint and secret.	Registration and idea creation.
BN254 G1	Pure Python curve arithmetic	Group operations for commitments and proofs.	ZKP and sharing.
Pedersen commitment	C = sG + rH with s = Poseidon(fingerprint, secret)	Hiding commitment used by the proof.	Registration, proof generation.
Schnorr proof	Fiat-Shamir non-interactive over BN254 G1	Knowledge of the commitment opening.	Proof generation and verification.
Argon2id	argon2-cffi	Password hashing.	Sign-up, login, password change.
JWT	python-jose	Access and refresh token claims.	Authentication.
AES-256-GCM	cryptography	Chunked file and evidence encryption.	AQVL1 storage containers.
Fernet	cryptography	Idea descriptions, notes, commitment secrets.	Ideas and internal secret storage.

The commitment

For each registration the server computes a Poseidon digest of the file fingerprint and a fresh random secret, then commits to it as sG + rH. The secret and blinding factor are stored encrypted; neither is returned to the browser. The commitment is public: it can be shown, shared, and checked without revealing the opening.

```
s = Poseidon254(fingerprint mod p, secret)
C = sG + rH
proof: knowledge of (s, r) such that C = sG + rH
```

The proof binds the registration reference, fingerprint, commitment, and a one-time nonce into the Fiat-Shamir challenge. That binding is what prevents a valid proof from being replayed against another record. The scheme identifiers are AUTHENTIQ-POSEIDON254-COMMITMENT-V1 and AUTHENTIQ-SCHNORR-BN254-G1-V1.

Verification, step by step

01 Validate the proof package shape and scheme identifier.

02 Check that the registration reference matches the verification request.

03 Compare the claimed fingerprint with the stored registration fingerprint.

04 Check that the nonce matches the unused challenge and has not expired.

05 Recompute the Fiat-Shamir challenge from the bound public signals.

06 Evaluate the Schnorr equation against the stored commitment.

07 Record the result, increment usage, and write an audit entry.

What the default proof does and does not establish

ESTABLISHES	DOES NOT ESTABLISH
The prover knows the opening of the stored commitment.	That the prover is the legal author.
The proof is fresh and bound to this registration and nonce.	That the file is original or non-infringing.
The presented fingerprint matches the registered fingerprint, checked outside the circuit.	That the fingerprint equality is constrained inside a zero-knowledge circuit.
A verifier can check the claim without receiving the file or secret.	That the ledger timestamp is independently witnessed.

Symmetric encryption and secrets

Completed files are written as AQVL1 containers: chunked AES-256-GCM with a per-chunk nonce and authentication tag. The encryption key is derived deterministically from AUTHENTIQ_SECRET_KEY. Idea descriptions, notes, and commitment secrets use Fernet text fields. The distinction matters: AQVL1 protects large binary files, Fernet protects small database text values.

The secret key is the root of both token signing and encryption-key derivation. Rotating it without a migration would make old files unreadable and old tokens invalid. In production, use a managed secret with a documented rotation and re-encryption plan.

Passwords and tokens

- Passwords are hashed with Argon2id and a per-password salt; plaintext is never stored or logged.

- Access tokens last 45 minutes and carry user, session, role, and organization claims.
- Refresh tokens last 14 days, are stored hashed, and rotate on every refresh. Reuse of a consumed token fails.
- Password reset tokens last 30 minutes and are single-use.
- A password change revokes every session.

THE SENTENCE TO KEEP

The default proof proves knowledge of the commitment opening. File-hash equality is verified separately before proving and during verification; it is not constrained inside the default circuit.

The optional Circom circuit

circuits/ownership.circom proves commitment = Poseidon(fingerprint mod p, secret) with Circom 2.1.6, Circomlib 2.0.5, and snarkjs 0.7.5. It is a reference for a future Groth16 integration and is not wired into runtime proof generation or verification. The shipped runtime proof remains the Schnorr scheme.

CHAPTER 34 · TECHNOLOGY

Blockchain and ledger tooling

The default ledger is a local append-only JSONL simulation. Polygon Amoy is an optional adapter that writes real testnet transactions. The product never calls the simulation a blockchain.

The simulation record

FIELD	MEANING
block_number	Monotonic position in the local chain.
prev_hash	Record hash of the previous entry.
record_hash	SHA-256 over the canonical record body.
payload_digest	SHA-256 of the canonical event payload.
tx_id	Application-level transaction identifier.
type	registration, proof, transfer, team, dispute, idea, and related events.
actor_id and actor_ref	Who caused the event.
entity_type and entity_id	What the event is about.
timestamp	UTC creation time.
network	AUTHENTIQ Local Ledger by default.

How chain verification works

01 Read the JSONL file in order.

02 Recompute each record hash from its canonical body.

03 Check that each prev_hash equals the previous record hash.

04 Check that each block number increases by one.

05 Recompute each payload digest and compare it with the stored value.

06 Report the first mismatch with its block number and reason.

Appending calls fsync after writing the line, which reduces the chance of losing the tail on a crash. It does not make the file immutable. An operator with filesystem access can rewrite the whole chain, so verification is a detection mechanism, not a distributed trust guarantee.

The Polygon Amoy adapter

When AUTHENTIQ_LEDGER_MODE=polygon and the RPC URL, private key, contract address, and chain ID are configured, the adapter calls recordEvent(recordId, recordType, payloadDigest, entityId) on the Solidity registry, waits for a receipt, and returns the real transaction hash and block number. If configuration or connectivity fails, the operation fails with a clear error; it never silently falls back to simulation.

COMPONENT	VERSION OR ROLE	WHERE
Solidity	0.8.24	blockchain/contracts/AuthentiqRegistry.sol
Hardhat	2.22	Compile, test, and deploy.
Polygon Amoy	Chain ID 80002	Public testnet target.
web3.py	Optional package	JSON-RPC and contract calls in the adapter.
eth-account	Optional package	Transaction signing.
Amoy explorer	amoy.polygonscan.com	Human inspection of testnet transactions.

Simulation, testnet, and mainnet are not the same

PROPERTY	LOCAL SIMULATION	POLYGON AMOY	PUBLIC MAINNET
Who can rewrite history	Anyone with filesystem access	Amoy validators and the contract writer	Mainnet consensus
Cost	Free	Testnet gas	Real gas
Privacy	Local file only	Public testnet transaction	Public permanent transaction
Independence	None	Third-party testnet	Strong public settlement
Used by default	Yes	No, opt-in	No

THE HONEST LABEL

A local hash-chained file is an append-only ledger simulation. It is tamper-evident and useful for a prototype. It is not distributed consensus, and the interface must never imply otherwise.

Enabling the adapter

01 Deploy AuthentiqRegistry.sol with Hardhat.

02 Authorize the backend signing address as a writer if it differs from the deployer.

03 Install web3 and eth-account in the backend environment.

04 Set ledger mode to polygon, plus RPC URL, private key, contract address, and chain ID 80002.

05 Restart the API and check GET /api/ledger/status for connectivity and mode.

Amoy is a public testnet. Transactions are real testnet transactions, not mainnet settlement. The optional Circom circuit is also not wired into runtime: it proves the Poseidon relation for a future Groth16 integration, while the shipped proof remains Schnorr.

CHAPTER 35 · TECHNOLOGY

Files, PDFs, and QR codes

The file pipeline is where most trust products become vague. Here is the exact path from a selected file to an encrypted container, a certificate, and a portable evidence packet.

Upload and storage path

01 The browser slices the file into chunks of at most 8 MB.

02 Each chunk is hashed and posted; the server checks the hash and size, writes it temporarily, and records the digest.

03 Completion re-hashes the chunks, assembles the file, compares the overall SHA-256, and runs the scanner gate.

04 The assembled file is encrypted into an AQVL1 container with chunked AES-256-GCM, then the temporary chunks are deleted.

05 The database stores the registration, metadata, digests, commitments, and the storage path.

The AQVL1 container

MAGIC AND VERSION	AQVL1 prefix, so the format can evolve explicitly.
CIPHER	AES-256-GCM with a per-chunk nonce and authentication tag.
KEY	Derived deterministically from AUTHENTIQ_SECRET_KEY.
INTEGRITY	Each chunk is authenticated; a modified container fails decryption.
TEMPORARY STATE	Plaintext chunks exist between chunk upload and completion, then are deleted.
EVIDENCE	Dispute evidence uses the same container pattern under storage/evidence.

The certificate

Certificates are generated on demand with ReportLab and are not cached. The PDF uses the house design system and contains the registration reference, title, owner display name, SHA-256 fingerprint, Poseidon and Pedersen commitments, registration timestamp, ledger transaction, a simulation or chain-mode label, and a QR code that points to the public verification page.

- The QR code is useful only while the verification link or record remains accessible.
- The certificate states what the record is, not what a court should conclude.

- A version certificate and a team certificate use the same generators with their own fields.
- Regenerating after a change produces a current document; there is no stale cached copy.

The evidence packet

FILE	CONTENTS	WHY IT IS THERE
manifest.json	Canonical registration, file metadata, ledger receipt, proofs, trust score and factors, self-hash.	The machine-readable core of the packet.
certificate.pdf	The rendered certificate for the record.	A human-readable artifact.
ledger-receipt.json	Block number, record hash, previous hash, payload digest.	Lets a reviewer compare the record with the chain.
file.sha256.txt	The digest in sha256sum-compatible form.	Offline byte verification.
VERIFY.txt	Step-by-step instructions, including recomputing the manifest hash.	Makes the packet self-describing.

WHAT THE PACKET DOES NOT CONTAIN

The original file, the commitment secret, the password, or a private key. It is evidence about a record, not a copy of the vault.

Storage directories

PATH	CONTENTS
storage/uploads/	Encrypted AQVL1 containers and temporary chunk directories.
storage/evidence/	Encrypted dispute evidence grouped by dispute.
storage/ledger.jsonl	The append-only simulation ledger.
backend/authentiq.db	SQLite fallback database when PostgreSQL is not configured.

Retention and backup

There is no automatic retention or deletion policy. Files remain until an operator removes them. Because the database stores paths rather than bytes, a restore that only covers PostgreSQL leaves records pointing at missing files. Back up the database, the upload and evidence directories, and ledger.jsonl together, and test a restore before relying on it.

CHAPTER 36 · TECHNOLOGY

Infrastructure, documentation, and configuration

The last technical chapter covers how the product is served, tested, configured, and documented. It is the layer that makes the rest reproducible.

Serving and orchestration

TOOL	ROLE	NOTES
Vite	Development server and production bundle.	Port 5173 in development; /api proxies to 8000.
Nginx	Static SPA serving, history fallback, /api reverse proxy.	Port 8080 in Compose.
Docker	Backend and frontend images.	Reproducible runtime.
Docker Compose	PostgreSQL 16, backend, frontend, health checks, volumes.	Backend waits for the database health check.
Uvicorn	ASGI server.	Single worker recommended with the in-process limiter.
pytest	Backend verification.	76 tests across auth, uploads, ZKP, ledger, audit, teams, transfers, disputes, dashboard, organization member provisioning, and audit chain concurrency.
TypeScript and Vite build	Frontend verification.	tsc --noEmit then vite build.
Hardhat	Optional Solidity tooling.	Compile, test, deploy the registry.

Commands

```
cd backend
.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m app.seed

cd frontend
npm install
npm run dev
npm run build
```

Configuration reference

VARIABLE	PURPOSE	DEFAULT
AUTHENTIQ_SECRET_KEY	JWT signing and encryption key derivation.	Development placeholder
AUTHENTIQ_DATABASE_URL	SQLAlchemy URL.	SQLite at backend/authentiq.db
AUTHENTIQ_STORAGE_DIR	Storage root.	storage/
AUTHENTIQ_UPLOAD_DIR	Encrypted uploads.	storage/uploads
AUTHENTIQ_EVIDENCE_DIR	Encrypted evidence.	storage/evidence
AUTHENTIQ_LEDGER_FILE	Simulation ledger path.	storage/ledger.jsonl
AUTHENTIQ_LEDGER_MODE	simulation or polygon.	simulation
AUTHENTIQ_CHAIN_RPC_URL	Polygon RPC endpoint.	Empty
AUTHENTIQ_CHAIN_PRIVATE_KEY	Writer key for the registry.	Empty
AUTHENTIQ_CHAIN_CONTRACT_ADDRESS	Registry address.	Empty
AUTHENTIQ_CHAIN_ID	Amoy chain ID.	80002
AUTHENTIQ_ZKP_CHALLENGE_MINUTES	Challenge lifetime.	10
AUTHENTIQ_ZKP_VERIFICATION_HOURS	Share lifetime bound.	72
AUTHENTIQ_MAX_FILE_SIZE_MB	Upload cap.	200
AUTHENTIQ_MAX_CHUNK_SIZE_MB	Chunk cap.	8
AUTHENTIQ_CORS_ORIGINS	Allowed browser origins.	Local Vite origins
AUTHENTIQ_PUBLIC_APP_URL	Verification link base.	http://localhost:5173
AUTHENTIQ_SEED_DEMO	Create demo accounts at startup.	false

Documentation set

DOCUMENT	WHAT IT COVERS
README.md	Product summary, implemented capabilities, quick start, demo accounts, stack, limitations.
docs/ARCHITECTURE.md	Components, flows, data model, ledger, audit, authorization, deployment, failure behavior.
docs/TECH_STACK.md	Versions, rationale, storage formats, cryptography, intentional omissions.
docs/API.md	Endpoint tables, payloads, status codes, rate limits, integrity and trust endpoints.
docs/SECURITY.md	Threat model, controls, residual risks, disclosure policy.
docs/ZKP.md	Exact proof statement, protocol, verification, limitations.
blockchain/README.md	Contract, Hardhat, Amoy deployment.
circuits/README.md	Optional Circom circuit and why it is not runtime.

Deliberate omissions

- No external identity provider; identity is local.
- No email delivery provider; development tokens are returned outside production.
- No object storage SDK; files live on the local filesystem.
- No Redis or distributed cache; rate limiting is in-process.
- No background task queue; certificates and hashing are synchronous.
- No production migration pipeline.
- No on-chain proof verification and no Groth16 runtime integration.

WHY LIST OMISSIONS IN A HANDBOOK

Because a reader who discovers them later will distrust everything else. Naming them here makes the boundary of the product legible and gives the next engineer a real backlog.

PART V

TRUST AND OPERATIONS

The security boundary, the complete API surface, the commands that run the product, the limitations that must be stated out loud, and a checklist anyone can use to reproduce the important claims.

This part is written for the person who has to defend the product to a client, an auditor, or another engineer. It names what is protected, what is not, and how to check.

IN THIS PART

37 Security model and trust boundaries
38 API map, status codes, and limits
39 Running AUTHENTIQ
40 Honest limitations
41 Verification checklist and glossary

PART V OF V

CHAPTER 37 · TRUST

Security model and trust boundaries

Security in AUTHENTIQ is not a single feature. It is the combination of password hashing, token rotation, backend authorization, encrypted storage, integrity checks, and an append-only audit trail.

Implemented controls

CONTROL	IMPLEMENTATION	WHAT IT PROTECTS
Password hashing	Argon2id with per-password salts.	Stolen database does not reveal passwords.
Access tokens	JWT, 45 minutes, signed with the application secret.	Short-lived API authorization.
Refresh tokens	14 days, stored hashed, rotated on use.	Longer sessions without long-lived bearer tokens.
Session revocation	Session rows can be revoked individually or all at once.	Lost devices and suspected compromise.
Login lockout	Temporary lock after repeated failures.	Online password guessing.
Rate limiting	Sliding window per scope.	Brute force, proof spam, share enumeration.
Backend authorization	Ownership, organization, and role checks on every protected resource.	Horizontal and vertical privilege escalation.
Encrypted files	AES-256-GCM in AQVL1 containers.	Files at rest on disk.
Encrypted secrets	Fernet for idea text and commitment secrets.	Sensitive database fields.
Integrity checks	Chunk hashes, overall SHA-256, scanner gate.	Corruption and tampering in transit.
Audit chain	Hash-chained audit entries and a verification endpoint.	Silent changes to security history.
Security headers and CORS	Configured in the API and proxy.	Common browser attack surface.

Threats and residual risk

THREAT	CONTROL	RESIDUAL RISK
Credential stuffing	Argon2id, lockout, login rate limit.	Distributed attempts across many IPs can still succeed slowly.
Token theft via XSS	Short access lifetime, no third-party scripts, React escaping.	localStorage is readable by injected script; a strict CSP is still required.
Horizontal access	Resource-level ownership and organization checks.	A logic bug in a new endpoint could bypass the helper.
Replay of a proof	One-time nonce, expiry, registration binding, usage limits.	A leaked valid package can be used within its limits if the link is shared.
Insider file access	Encryption at rest, access checks.	An operator with the secret key and filesystem can decrypt.
Ledger rewrite	Hash chain and fsync.	Filesystem access allows a full rebuild; not distributed consensus.
Database rollback	Audit entries and chain verification.	A consistent rollback of both data and audit entries may not be detectable.
Upload abuse	Extension allow-list, size cap, scanner gate, rate limits.	A zero-day in a parser or scanner remains possible.
Enumeration	404 for hidden resources, scoped queries, rate limits.	Timing and error differences are always a residual channel.

What no role can do

- Read another user's private registration without a share link or a reviewer role that covers it.
- Resolve a dispute that they are a party to as if they were neutral, unless they hold the verifier role or an organization admin scope over the record.
- Recover a commitment secret or a file password from the API; secrets are encrypted and never returned.
- Turn a local simulated ledger record into a public blockchain transaction without configuring the Polygon adapter.
- Bypass rate limits or account lockout by changing a route in the browser.

The trust boundaries

BROWSER	Untrusted. It may be modified, extended, or controlled by an attacker. Every value it sends is validated.
API	Trusted application boundary. It holds the secret key, database credentials, and encryption key derivation.
DATABASE	Trusted infrastructure. Operator access is equivalent to full data access.
FILESYSTEM	Trusted infrastructure. Holds encrypted files, evidence, and the simulation ledger.
POLYGON AMOY	External, optional. A testnet is independent but not mainnet settlement.
EMAIL AND MFA PROVIDERS	Absent. The product reports them as unavailable instead of simulating them.

What the product does not claim

- That a hash proves authorship or originality.
- That a local ledger is a blockchain or distributed consensus.
- That a proof establishes legal ownership.
- That the platform is a court, arbitrator, or custody service.
- That unavailable email, MFA, or AI classification is active.

THE DEFENSIBLE CLAIM

AUTHENTIQ records exact bytes, binds them to a commitment, proves knowledge of the opening, and preserves a hash-chained history with explicit labels. Every part of that sentence is implemented and testable.

CHAPTER 38 · TRUST

API map, status codes, and limits

Everything the browser does goes through /api. The OpenAPI document at /api/docs is the live contract; this chapter is the human map.

Conventions

BASE PATH	/api. The frontend uses VITE_API_URL and defaults to /api.
AUTHENTICATION	Authorization: Bearer access_token. Refresh uses the refresh token in the JSON body.
CONTENT TYPES	application/json for most requests; multipart/form-data for chunks and evidence.
ERRORS	A JSON detail field. Status codes follow the table below.
PAGINATION	Limit and offset query parameters where a list can grow.
DOWNLOADS	Authenticated fetch, returned as a blob; no public file URLs.

System and authentication

METHOD	PATH	PURPOSE
GET	/health	Status, version, ledger mode, proof scheme.
GET	/docs, /redoc, /openapi.json	Interactive and machine-readable API documentation.
POST	/auth/register	Create an individual or organization account.
POST	/auth/login	Sign in and create a session.
POST	/auth/refresh	Rotate the refresh token and issue a new access token.
POST	/auth/logout	Revoke one session or all sessions.
GET, PATCH	/auth/me	Read or update the current profile.
POST	/auth/forgot-password	Create a reset token.
POST	/auth/reset-password	Consume a reset token and set a new password.
POST	/auth/change-password	Change the password and revoke all sessions.
GET	/auth/sessions	List active sessions.
DELETE	/auth/sessions/{session_id}	Revoke one session.

Uploads, fingerprints, and registrations

METHOD	PATH	PURPOSE
POST	/uploads/initiate	Start a chunked upload and create a chunk plan.
POST	/uploads/{upload_id}/chunk	Upload and verify one chunk.
POST	/uploads/{upload_id}/complete	Assemble, scan, hash, and encrypt.
GET	/uploads/{upload_id}	Upload status.
POST	/fingerprints/generate	Compute SHA-256 over a supplied file.
POST	/fingerprints/verify	Compare a hash with a fingerprint.
POST	/registrations	Create a registration from a completed upload.
POST	/registrations/duplicate-check	Check a fingerprint before upload.
GET	/registrations	List accessible registrations.
GET	/registrations/{identifier}	Registration detail.
GET	/registrations/{identifier}/file	Decrypt and download the original.
GET	/registrations/{identifier}/certificate	Download the PDF certificate.
GET	/registrations/{identifier}/trust-score	Explainable evidence score.
GET	/registrations/{identifier}/evidence-packet	Download the ZIP packet.
GET	/registrations/{identifier}/verification-state	Ledger record and recent proofs.

Proofs, sharing, ideas, and versions

METHOD	PATH	PURPOSE
POST	/zkp/challenge	Create a one-time nonce.
POST	/zkp/generate	Generate a proof from a local fingerprint.
POST	/zkp/verify	Verify a proof.
GET	/zkp/proofs/{registration_id}	List recent proofs.
POST	/sharing/create	Create a verification link and package.
GET	/sharing	List links created by the user.
DELETE	/sharing/{verification_id}	Revoke a link.
GET, POST	/public/verify/{verification_id}	Read disclosure-limited metadata or submit a proof.
POST, GET	/ideas	Create or list ideas.
GET	/ideas/{identifier}	Idea detail.
POST	/ideas/{identifier}/revisions	Add an encrypted revision.
PATCH	/ideas/{identifier}/notes	Update encrypted notes.
POST	/ideas/{identifier}/reveal	Reveal and record the event.
GET	/ideas/{identifier}/certificate	Idea certificate.
GET	/ideas/{identifier}/evidence	Idea evidence package.
POST	/versions	Add a version from a completed upload.
GET	/versions/registration/{registration_id}	List versions.
GET	/versions/compare	Compare two versions.
GET	/versions/{version_id}	Version detail.
GET	/versions/{version_id}/file	Download a version.
GET	/versions/{version_id}/certificate	Version certificate.

Collaboration, governance, and trust

METHOD	PATH	PURPOSE
POST, GET	/teams	Create or list team projects.
GET	/teams/{project_id}	Project detail.
POST	/teams/{project_id}/invite	Invite a member and issue an invite code.
POST	/teams/{project_id}/join	Accept an invitation.
POST	/teams/redeem	Redeem an invite code and join the issuing project.
POST	/teams/{project_id}/contributions	Submit a contribution.
PATCH	/teams/{project_id}/contributions/{id}/approve	Approve a contribution.
POST	/teams/{project_id}/ownership/propose	Propose percentages.
POST	/teams/{project_id}/ownership/approve	Approve an allocation.
GET	/teams/{project_id}/ownership	Allocation state.
GET	/teams/{project_id}/certificate	Team certificate.
POST, GET	/transfers	Propose or list transfers.
GET	/transfers/history/{registration_id}	Append-only transfer history.
POST	/transfers/{id}/accept, reject, cancel, confirm	Transfer transitions.
POST, GET	/disputes	Open or list disputes.
GET	/disputes/{dispute_id}	Dispute detail.
POST	/disputes/{dispute_id}/evidence	Upload encrypted evidence.
GET	/disputes/{dispute_id}/evidence/{evidence_id}	Download evidence.
POST	/disputes/{dispute_id}/status	Update or resolve a dispute.
GET	/disputes/{dispute_id}/report	JSON evidence report.
GET	/alerts, /alerts/{id}	List or inspect alerts.
PATCH	/alerts/{id}/status	Update the investigation state.
GET	/ledger/status, /ledger/transactions	Ledger status and transactions.
GET	/ledger/transactions/{tx_id}	Transaction detail.
POST	/ledger/audit	Verify the active ledger.
GET	/integrity/overview	Full integrity report.
GET	/notifications	Inbox.
PATCH	/notifications/{id}/read	Mark one read.
POST	/notifications/read-all	Mark all read.

METHOD	PATH	PURPOSE
DELETE	/notifications/{id}	Dismiss.
GET	/audit-logs, /audit-logs/verify	Audit list and chain verification.
GET	/search?q=	Cross-entity search.
GET, PATCH	/settings/organization	Organization details, members, invitations, and rename.
POST	/settings/organization/members	Provision a member account with a generated password.
POST	/settings/organization/members/{id}/password	Rotate a member password and revoke sessions.
GET, POST	/settings/organization/invitations	List and issue organization invitation codes.
POST	/settings/organization/invitations/accept	Redeem an organization invitation code.
DELETE	/settings/organization/invitations/{id}	Revoke an unaccepted invitation.
GET	/settings/organization/registrations	Organization registrations.
GET	/settings/security	Security capability report.
GET	/dashboard/summary, /dashboard/activity	Dashboard data.
GET	/public/records	Public records registry.
GET	/admin/overview, /admin/users, /admin/organizations	Site administration overview and lists.
GET, PATCH	/admin/users/{id}, /admin/disputes/{id}	Site administration writes.

Status codes

STATUS	MEANING
400	Invalid state or bad request.
401	Missing, expired, or invalid token.
403	Authenticated but not authorized.
404	Not found, or hidden by access control.
409	State conflict, duplicate, replay, or usage limit.
410	Expired or revoked resource.
413 / 415	File too large or unsupported extension.
422	Validation failure or integrity mismatch.
423	Account temporarily locked.
429	Rate limit exceeded; inspect Retry-After.
500	Unexpected server error.

Rate limits

SCOPE	LIMIT	WINDOW
Login	8 per IP and email	5 minutes
Registration	10 per IP	1 hour
Proof generation	40 per user	1 hour
Share creation	30 per user	1 hour
Public verification	30 per IP	10 minutes

THE LIVE CONTRACT WINS

This table is a map, not the schema. Before integrating, read /api/openapi.json and the docs/API.md reference for exact payload fields and validation rules.

CHAPTER 39 · OPERATIONS

Running AUTHENTIQ

The product runs locally with Python and Node. The current workspace uses a hosted Neon database; SQLite remains the zero-setup fallback. Docker Compose provides a production-like topology.

Prerequisites

- Python 3.11 or newer. The container uses 3.12; local verification used 3.14.
- Node.js 20 or newer and npm.
- Optional: Docker Desktop for the Compose topology.
- Optional: a PostgreSQL URL such as Neon for shared state.

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000
```

The API runs at http://localhost:8000. Health is at /api/health, Swagger at /api/docs, ReDoc at /api/redoc. On startup the application creates directories and tables, and optionally seeds demo accounts when AUTHENTIQ_SEED_DEMO is true.

Frontend

```
cd frontend
npm install
npm run dev
```

Vite runs at http://localhost:5173 and proxies /api to http://localhost:8000. The landing page is /, the workspace sign-in is /login. A production build runs tsc --noEmit followed by vite build; the static output can be served by Nginx with history fallback.

Database

Set AUTHENTIQ_DATABASE_URL in backend/.env to use PostgreSQL. The active workspace points at a Neon pooled connection with sslmode=require and channel_binding=require. Without the variable, the backend uses backend/authentiq.db. Tables are created at startup; there is no migration history yet.

Demo accounts

ROLE	EMAIL	PASSWORD
Organization admin	demo@authentiq.dev	Authentiq!Demo2026
Verifier	verifier@authentiq.dev	Authentiq!Verify2026
Creator	maya@authentiq.dev	Authentiq!Team2026

The seed creates only these three logins. It creates no registrations, ideas, teams, alerts, transfers, disputes, shares, ledger events, or audit entries. The workspaces start empty by design, so the first dashboard is a true zero state.

Verification commands

```
cd backend
.venv\Scripts\python.exe -m pytest -q

cd frontend
npm run build
```

The backend suite currently passes 76 tests. The frontend build passes TypeScript checking and Vite bundling; the only expected warning is Vite's chunk-size advisory.

Docker Compose

```
copy .env.example .env
docker compose up --build
docker compose exec backend python -m app.seed
```

The Compose topology runs PostgreSQL 16, the backend, and an Nginx-served frontend. Nginx listens on 8080, proxies /api to the backend on 8000, and serves the SPA with history fallback. PostgreSQL and the storage directory use named volumes. The backend waits for the database health check before starting.

Health check

```
GET http://localhost:8000/api/health
{ status, version, ledger_mode, zkp_scheme, database }
```

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
401 on every request	Expired access token or missing refresh token.	Sign in again; check that localStorage holds both tokens.
CORS error in the browser	Origin not in AUTHENTIQ_CORS_ORIGINS.	Add the exact scheme, host, and port.
Database connection timeout	Neon paused, wrong URL, or network block.	Wake the database, verify the URL, check port 5432 access.
Upload completes but registration fails	Upload not owned by the user or not completed.	Re-initiate and complete the upload before registering.
Proof generation disabled	Local hash does not match the registered fingerprint.	Select the exact original file; even one changed byte changes the hash.
Share link returns 410	Expired or revoked.	Create a new link with a longer expiry.
Ledger chip says simulation	Default mode.	That is expected unless the Polygon adapter is configured.

CHAPTER 40 · TRUST

Honest limitations

These are the boundaries a serious operator must be able to say out loud. None of them is hidden in the interface, and none should be omitted from a sales conversation.

The ten limits that matter

LIMITATION	WHAT IT MEANS	WHAT TO DO ABOUT IT
Local ledger is simulated	It detects local tampering but is not distributed consensus and can be rewritten by an operator.	Use the Polygon adapter when independent timestamp evidence is required.
Proof scope is narrow	The default proof proves knowledge of the commitment opening; file equality is checked outside the circuit.	State the claim precisely; do not call it authorship proof.
No legal determination	Registration is priority evidence, not a court judgment or copyright registration.	Keep the certificate and packet as supporting evidence; consult counsel.
No email delivery	Password reset returns a development token, and team invitations are delivered as a one-time invite code shown to the owner.	Configure a provider before production and send the code by mail.
No MFA or passkeys	Account security depends on password, token hygiene, and device security.	Add TOTP or WebAuthn before handling high-value accounts.
Files are local	Encrypted files and the ledger are on disk, not object storage.	Use encrypted volumes, off-box backups, and a restore test.
No migration history	Tables are created from metadata at startup; schema changes are not versioned.	Generate an Alembic baseline and run migrations separately.
In-process rate limiting	Multiple workers do not share counters.	Move limits to Redis or the edge before scaling horizontally.
No background workers	Hashing, encryption, and certificate generation are synchronous request work.	Add a queue for large files and report generation.
AI analysis is unavailable	No provider credentials are configured; the UI reports that honestly.	Configure a real provider or leave the analysis action hidden.

Claims to avoid

- Do not say the product is a blockchain by default. Say it uses an append-only local ledger with hash chaining, with an optional Polygon testnet adapter.
- Do not say a proof proves authorship. Say it proves knowledge of the commitment opening and that the file hash is checked separately.
- Do not say registration creates a copyright. Say it creates dated priority evidence.
- Do not say the platform is court-admissible. Say it produces portable evidence that a reviewer can inspect.
- Do not say files are stored in the cloud. Say relational state is hosted and encrypted files and the ledger are on local disk in this deployment.

What is already stronger than a prototype norm

- Real SHA-256, Poseidon, BN254, Pedersen, and Schnorr code, not mocked responses.
- Real chunk verification, encryption, certificates, and evidence packets.
- Backend-enforced roles with tests for authorization failures.
- A test suite that deliberately tampers with proofs, ledgers, and audit chains.
- Honest availability flags for missing email, MFA, passkeys, and AI analysis.
- A deterministic, explainable trust score instead of a single unexplained badge.

THE STANDARD TO HOLD

The product should be described at the strength of its weakest true sentence, not its strongest marketing sentence. That is how trust products survive contact with a skeptical reviewer.

CHAPTER 41 · APPENDIX

Verification checklist and glossary

You do not have to take this handbook on faith. These are the checks you can run to confirm the important claims, followed by the vocabulary needed to discuss them precisely.

Ten checks you can run

01 Hash the same file twice. The SHA-256 value must be identical, and changing one byte must change it completely.

02 Upload a file with a corrupted chunk. The API must return 422 and refuse to complete the upload.

03 Register a file, then register the identical bytes again. The second registration must succeed but create duplicate alerts and notifications.

04 Generate a proof with a different file. The proof must be refused or fail verification because the fingerprint does not match.

05 Reuse a verification package after its use limit. The API must return 409, not a success.

06 Open an expired or revoked share. The API must return 410.

07 Edit one byte in ledger.jsonl and run the audit. Verification must report a mismatch at the edited record.

08 Edit one audit entry and run the audit-chain check. The chain must fail at the edited entry.

09 Sign in as a creator and request another user's registration. The API must return 404 or 403, never the record.

10 Change a password and reuse an old refresh token. The refresh must fail because every session was revoked.

CHECK	EXPECTED RESULT	WHERE TO LOOK
Chunk integrity	422 and no recorded chunk	Upload endpoint response
Duplicate detection	Registration plus alerts and notifications	Alerts page and registration result
Proof mismatch	Proof refused or invalid	Verification page and proof list
Share reuse	409	Public verification endpoint
Expired share	410	Public verification endpoint
Ledger tamper	First mismatch with block number	Integrity center and ledger audit
Audit tamper	First mismatch with entry id	Audit chain verify
Cross-account access	403 or 404	Registration detail endpoint
Revoked session	401 on refresh	Auth refresh endpoint
Invite code reuse	404 on the second redeem	Teams redeem endpoint

Glossary

TERM	MEANING IN AUTHENTIQ
Access token	A 45-minute JWT used as a bearer credential for API requests.
AQVL1	The versioned encrypted container format for uploaded files and evidence.
Audit chain	A hash-chained log of security-relevant actions, separate from the ledger.
Commitment	A hiding commitment C = sG + rH that binds a fingerprint and secret without revealing them.
Creator	The default role for every public registration; scope is the user's own resources.
dHash-64	A 64-bit perceptual image hash used for near-duplicate detection.
Evidence packet	A ZIP with manifest, certificate, ledger receipt, file digest, and verification instructions.
Fingerprint	The SHA-256 digest of the exact file bytes.
Fiat-Shamir	The transform that makes the Schnorr proof non-interactive by hashing the challenge.
Integrity center	The diagnostic page that recomputes both chains, storage presence, linkage, and duplicates.
Ledger	The append-only JSONL event history; simulated locally, optionally anchored on Polygon Amoy.
Manifest	The canonical JSON description of an evidence packet, including its own SHA-256.
Org admin	The organization owner role; adds organization records and in-organization dispute review.
Ownership DNA	A deterministic grid derived from the fingerprint bits.
Pedersen commitment	The BN254 commitment used by the Schnorr proof.
Poseidon-254	The BN254-friendly hash used to derive the committed value.
Proof package	A JSON package containing public signals and the proof, verifiable without the file.
Refresh token	A 14-day token stored hashed and rotated on use.
Registration	The dated record that binds a file fingerprint, commitments, metadata, and ledger event.
Schnorr proof	The non-interactive proof of knowledge of the commitment opening.
Silent sharing	A disclosure-limited verification link with an expiry and use limit.
Simulation ledger	The default local append-only ledger, clearly labelled as simulated.
Trust score	An explainable 0-100 evidence score with positive factors and penalties.
Verifier	A reviewer role that can see all disputes and all ledger events, but not all registrations.
ZKP	Zero-knowledge proof; in this product, the Schnorr proof over BN254 G1.

Closing

AUTHENTIQ is a working engineering prototype with real cryptography, real authorization, real storage encryption, and a real test suite. It is also a product with explicit boundaries: a local ledger, a narrow proof statement, no email, no MFA, and no legal authority. The value is not that it hides those boundaries. The value is that it gives a creator a precise, portable, privacy-preserving way to say: this is the work, this is when I recorded it, and this is how you can check.

KEEP THIS WITH THE PRODUCT

When the code changes, re-run the tests, update the version numbers, and re-check the claims in this handbook. A document that drifts from the implementation becomes the most expensive kind of fiction.

AUTHENTIQ
EVIDENCE, NOT ASSUMPTION.